



○ ○ ○ ○

**CSE338 – SOFTWARE TESTING,
VALIDATION, AND VERIFICATION**

SPRING 25

HRMS Report

Team 15





Prepared To :

Dr. Yasmine Afify

Eng. Amr Sayed

Prepared By:

Nadine Radwan Ali 22P0214

Hams Hassan Thabet 22P0253

Jana Sameh Samir 22P0237

Amr Adel 22P0311



Table of Contents

1. Introduction:	4
2. System Overview:	5
3. Test-Driven Development Methodology:	6
4. Unit Testing:	7
4.1 Test Suites:	7
4.1.1 Employee Management Test Suite:	7
4.1.2 Leave Request Test Suite:	11
4.1.3 Leave Management Test Suite:	13
4.1.4 Payroll Management Test Suite:	17
4.1.5 Performance Evaluation Test Suite:	20
4.1.6 Authentication Test Suite:	25
4.1.7 File Handler Test Suite:	27
5. White Box Testing:	30
5.1 Payroll Class:	31
5.1.1 Function 1: deductSalary(employee, amount)	31
5.1.2 Function 2: addBonus(employee, bonus)	32
5.1.3 Function 3: applyTax(employee, taxPercent)	33
5.1.4 Function 4: printPayrollSummary(employee)	34
5.2 LeaveManagement Class:	35
5.2.1 Function 1: validateLeaveRequest(employee, leave)	35
5.2.2 Function 2: calculateLeaveDays	37
5.2.3 Function 3: updateLeaveBalance	38
5.2.4 Function 4: displayLeaveRequest	39
5.3 PerformanceEvaluation Class:	40
5.3.1 Function 1: calculatePerformanceRate	40
5.3.2 Function 2: setPerformanceManually	41
5.3.3 Function 3: boostPerformance	42

5.3.4	Function 4: penalizePerformance	43
5.3.5	Function 5: displayPerformance	44
6.	GUI Testing:	45
	The Login page:	45
	HR Dashboard.....	47
	Add New Employee.....	48
	View all Employees page:	49
	Edit:.....	51
	Delete.....	53
	Performance	53
	Payroll	56
	Leaves	59
	The Employee's dashboard.....	60
7.	Integration Testing:.....	63
7.1	Integration Testing Checklist:.....	63
7.1.1	Login Integration:	63
7.1.2	Employee Dashboard Integration:	64
7.1.3	HR Dashboard Integration:	64
7.1.4	Employee Management Integration:	65
7.1.5	Leave Management Integration:	65
7.1.6	Payroll Integration:	66
7.1.7	Cross-Module Integration:.....	66
7.1.8	File & Backend Integration:.....	67
7.1.9	Error Handling & Edge Cases:	67
7.2	Integration Test Cases:	68
7.2.1	Test Case 1: HR Login and Add Employee:.....	68
7.2.2	Test Case 2: Employee Submits Leave Request and HR Approves:	70
7.2.3	Test Case 7: Apply Bonus and Verify Payroll UI:	71
7.2.4	Test Case 9: Delete Employee and Verify Login Failure:.....	72

7.2.5	Test Case 13: Leave Request Updates Leave Table:.....	73
7.2.6	Test Case 18: Full Workflow:	74
7.2.7	Test Case 20: Handle Invalid Leave Request (Past Date):	75
7.2.8	Test Case 21: Update Employee Details and Verify UI:	76
7.2.9	Test Case 22: Performance Review:	77
8.	Application UI:	78
9.	Conclusion:.....	84

1. Introduction:

The Human Resource Management System (HRMS) is a sophisticated software solution designed to optimize organizational HR processes, encompassing employee management, leave processing, payroll operations, performance evaluation, and secure authentication. Built with JavaFX for an intuitive and responsive user interface, Java for robust backend logic, and file-based persistence for efficient data storage, the HRMS delivers a seamless experience for both HR administrators and employees. The system supports role-based access, providing HR personnel with tools to manage employee records, approve leave requests, process payroll, and evaluate performance, while employees can view their profiles, submit leave requests, and track their performance metrics. This comprehensive functionality ensures operational efficiency, data accuracy, and user satisfaction, making the HRMS a critical tool for modern workforce management.

This Test-Driven Development (TDD) report outlines the rigorous testing strategy employed to validate the HRMS, ensuring its reliability, functionality, and usability. By adopting a TDD approach, the development process prioritized writing tests before coding, fostering a cycle of test creation, implementation, and iterative refinement. This methodology not only guaranteed high code quality but also enabled early defect detection, reducing maintenance efforts and enhancing system robustness. The report details four key testing phases—unit testing, white-box testing, GUI testing, and integration testing—each targeting specific system components, from individual classes to end-to-end workflows. Leveraging tools such as JUnit, Mockito, TestFX, and JaCoCo, the testing process achieved comprehensive coverage, addressing functional requirements (e.g., leave approval, payroll calculations) and non-functional aspects (e.g., UI responsiveness, error handling). The report also emphasizes the system's polished UI design, featuring responsive layouts, hover effects, and consistent styling, which enhance the user experience. Through detailed test cases, coverage metrics, and documentation, this report demonstrates the HRMS's adherence to high-quality software engineering standards, ensuring it meets organizational needs effectively.

2. System Overview:

The HRMS is a desktop application designed to manage HR-related tasks efficiently. Key features include:

- **Employee Management:** HR administrators can add, edit, and delete employee records, including details like ID, name, department, position, salary, leave balance, performance rating, and password.
- **Leave Management:** Employees can submit leave requests, which HR can approve or reject, with automatic updates to leave balances.
- **Payroll Processing:** Supports salary deductions, bonuses, and tax applications, with summaries for employee financial records.
- **Performance Evaluation:** Enables automatic rating calculations based on leave usage and manual adjustments, with boost and penalty options.
- **Authentication:** Role-based login system distinguishes between HR and employee access, ensuring secure data handling.
- **Data Persistence:** Uses flat files (employees.txt, leaves.txt) for storing employee and leave data, ensuring portability and simplicity.

The system caters to two primary user roles:

- **HR Administrators:** Manage all employee data, approve leaves, process payroll, and evaluate performance via a dedicated dashboard (**hr_dashboard.fxml**).
- **Employees:** View personal details, submit leave requests, and track leave status through an employee dashboard (**employee_dashboard.fxml**).

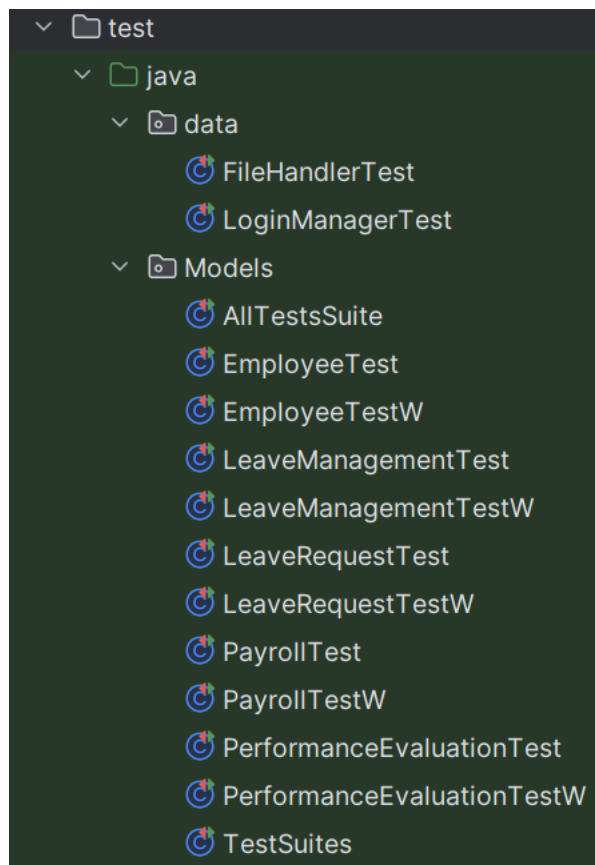
The HRMS is built to be user-friendly, with a modern JavaFX interface featuring color-coded buttons, responsive tables, and visual feedback (e.g., hover effects, drop shadows). Its modular design supports future enhancements, such as database integration or additional HR features.

3. Test-Driven Development Methodology:

The HRMS was developed using TDD, with test cases written before code implementation to ensure robust functionality and high coverage. The testing strategy leverages provided test files:

- **EmployeeTest.java, EmployeeTestW.java**
- **LeaveRequestTest.java, LeaveRequestTestW.java**
- **PayrollTest.java, PayrollTestW.java**
- **LeaveManagementTest.java, LeaveManagementTestW.java**
- **LoginManagerTest.java, FileHandlerTest.java**
- **PerformanceEvaluationTest.java, PerformanceEvaluationTestW.java**

FXML files (**login.fxml**, **hr_dashboard.fxml**, etc.), controller classes, and workspace.xml coverage data. The testing categories unit, white-box, GUI, and integration, Below is a concise overview of all test cases, organized by category, with key inputs, outputs, and purposes.



4. Unit Testing:

Unit testing validated individual components in isolation, focusing on the public methods of model classes (**Employee**, **LeaveRequest**, **Payroll**, **LeaveManagement**, **PerformanceEvaluation**), as well as **LoginManager** and **FileHandler**. Tests were written using JUnit, with Mockito for mocking dependencies and junit-jupiter-params for parameterized tests. Each test suite included setup and teardown methods to ensure a clean state, with assertions verifying expected behavior, including edge cases and error handling.

4.1 Test Suites:

Test suites for a Human Resource Management System (HRMS) are crucial for ensuring the software's reliability, functionality, and security. These suites encompass a variety of testing levels and types, including unit, integration, system, and acceptance testing, each with specific objectives. Functional testing ensures that modules like employee management, payroll, and benefits administration operate as expected, while non-functional testing, including performance and security testing, verifies the system's speed, stability, and data protection capabilities.

4.1.1 Employee Management Test Suite:

Test Class:

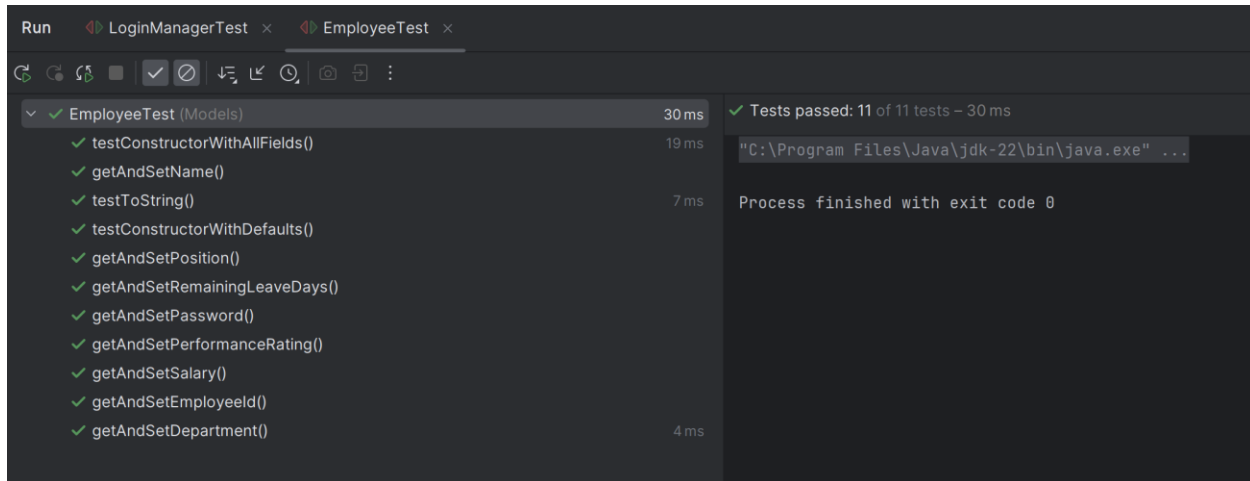
- EmployeeTest (model test)

Purpose: Validates employee data management, including model initialization and service operations like adding, retrieving, updating, and deleting employees.

Test Setup:

```
@BeforeEach
void setUp() {
    employee = new Employee( employeeId: 101, name: "John Doe", department: "IT",
    position: "Developer", salary: 5000.0, remainingLeaveDays: 25, performanceRating: 85.0, password: "secret123");
}
```


Test Results:



Test Cases:

- EmployeeTest:

1. testConstructorWithAllFields:

```
@Test
void testConstructorWithAllFields() {
    assertEquals( expected: 101, employee.getEmployeeId());
    assertEquals( expected: "John Doe", employee.getName());
    assertEquals( expected: "IT", employee.getDepartment());
    assertEquals( expected: "Developer", employee.getPosition());
    assertEquals( expected: 5000.0, employee.getSalary());
    assertEquals( expected: 25, employee.getRemainingLeaveDays());
    assertEquals( expected: 85.0, employee.getPerformanceRating());
    assertEquals( expected: "secret123", employee.getPassword());
}
```

- **Description:** Verifies full constructor initializes all fields.
- **Test:** new Employee(1, "Alice", "IT", "Developer", 5000.0, 30, 100.0, "pass1").
- **Assertions:** getEmployeeId() == 1, getName() == "Alice".

2. testConstructorWithDefaults:

```
@Test
void testConstructorWithDefaults() {
    Employee e = new Employee( employeeId: 202, name: "Jane", department: "HR", position: "Manager", salary: 6000.0, password: "pass456");
    assertEquals( expected: 202, e.getEmployeeId());
    assertEquals( expected: "Jane", e.getName());
    assertEquals( expected: "HR", e.getDepartment());
    assertEquals( expected: "Manager", e.getPosition());
    assertEquals( expected: 6000.0, e.getSalary());
    assertEquals( expected: 30, e.getRemainingLeaveDays()); // default
    assertEquals( expected: 100.0, e.getPerformanceRating()); // default
    assertEquals( expected: "pass456", e.getPassword());
}
```

- **Description:** Tests constructor omitting password, defaults leave days and rating.
- **Test:** new Employee(2, "Bob", "HR", "Manager", 4000.0, "pass2").
- **Assertions:** getRemainingLeaveDays() == 30, getPerformanceRating() == 100.0.

3. testSettersandGetters:

```
@Test
void getAndSetEmployeeId() {
    employee.setEmployeeId(202);
    assertEquals( expected: 202, employee.getEmployeeId());
}

@Test
void getAndSetName() {
    employee.setName("Alice");
    assertEquals( expected: "Alice", employee.getName());
}

@Test
void getAndSetDepartment() {
    employee.setDepartment("Marketing");
    assertEquals( expected: "Marketing", employee.getDepartment());
}

@Test
void getAndSetPosition() {
    employee.setPosition("Lead");
    assertEquals( expected: "Lead", employee.getPosition());
}

@Test
void getAndSetSalary() {
    employee.setSalary(7500.0);
    assertEquals( expected: 7500.0, employee.getSalary());
}
```

```

@Test
void getAndSetRemainingLeaveDays() {
    employee.setRemainingLeaveDays(18);
    assertEquals( expected: 18, employee.getRemainingLeaveDays());
}

@Test
void getAndSetPerformanceRating() {
    employee.setPerformanceRating(92.5);
    assertEquals( expected: 92.5, employee.getPerformanceRating());
}

@Test
void getAndSetPassword() {
    employee.setPassword("newPass");
    assertEquals( expected: "newPass", employee.getPassword());
}

@Test
void testToString() { assertEquals( expected: "101 - John Doe (Developer)", employee.toString()); }
}

```

4. testToString:

```

@Test
void testToString() { assertEquals( expected: "101 - John Doe (Developer)", employee.toString()); }
}

```

- **Description:** Verifies toString() format.
- **Test:** toString() on employee
- **Assertions:** Returns formatted string (e.g., "101 – Ahmed Amr (Developer)")

4.1.2 Leave Request Test Suite:

Test Class:

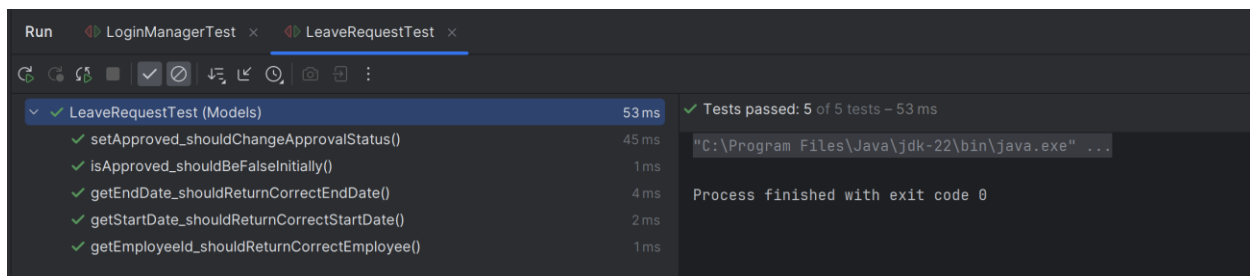
- LeaveRequestTest (model test)

Purpose: Verifies leave request creation and service operations like submission, status updates, and retrieval.

Test Setup:

```
@BeforeEach
void setUp() {
    testEmployee = new Employee(employeeld: 1001, name: "Nadine", department: "IT",
                                position: "Developer", salary: 6000.0, remainingLeaveDays: 30, performanceRating: 100.0, password: "pass123");
    start = LocalDate.of( year: 2025, month: 5, dayOfMonth: 10);
    end = LocalDate.of( year: 2025, month: 5, dayOfMonth: 15);
    leaveRequest = new LeaveRequest(testEmployee, start, end);
}
```

Test Results:



Test Cases:

- LeaveRequestTest:

1. testGetEmployeeId:

- **Description:** Verifies that the leave request returns the correct employee details.
- **Test:** getEmployeeId() on leaveRequest.
- **Assertions:** Returns testEmployee with name "Nadine" and department "IT".

2. testGetStartDate:

- **Description:** Checks that the leave request returns the correct start date.
- **Test:** getStartDate() on leaveRequest.
- **Assertions:** Returns LocalDate.of(2025, 5, 10).

3. testGetEndDate:

- **Description:** Checks that the leave request returns the correct end date.
- **Test:** getEndDate() on leaveRequest.
- **Assertions:** Returns LocalDate.of(2025, 5, 15).

4. testIsApproved:

- **Description:** Verifies that the leave (leave request?) is not approved by default.
- **Test:** isApproved() on leaveRequest.
- **Assertions:** Returns false.

5. testSetApproved:

- **Description:** Verifies that approval status can be changed.
- **Test:** setApproved(true) then setApproved(false) on leaveRequest.
- **Assertions:** isApproved() returns true after setting to true, and false after setting to false.

```

@Test
void getEmployeeId_shouldReturnCorrectEmployee() {
    assertEquals(testEmployee, leaveRequest.getEmployeeId());
    assertEquals( expected: "Nadine", leaveRequest.getEmployeeId().getName());
    assertEquals( expected: "IT", leaveRequest.getEmployeeId().getDepartment());
}

@Test
void getStartDate_shouldReturnCorrectStartDate() { assertEquals(start, leaveRequest.getStartDate()); }

@Test
void getEndDate_shouldReturnCorrectEndDate() { assertEquals(end, leaveRequest.getEndDate()); }

@Test
void isApproved_shouldBeFalseInitially() { assertFalse(leaveRequest.isApproved()); }

@Test
void setApproved_shouldChangeApprovalStatus() {
    leaveRequest.setApproved(true);
    assertTrue(leaveRequest.isApproved());

    leaveRequest.setApproved(false);
    assertFalse(leaveRequest.isApproved());
}

```

4.1.3 Leave Management Test Suite:

Test Class:

- LeaveManagement (model test)

Purpose: Validates LeaveManagement class methods for leave request validation and balance updates.

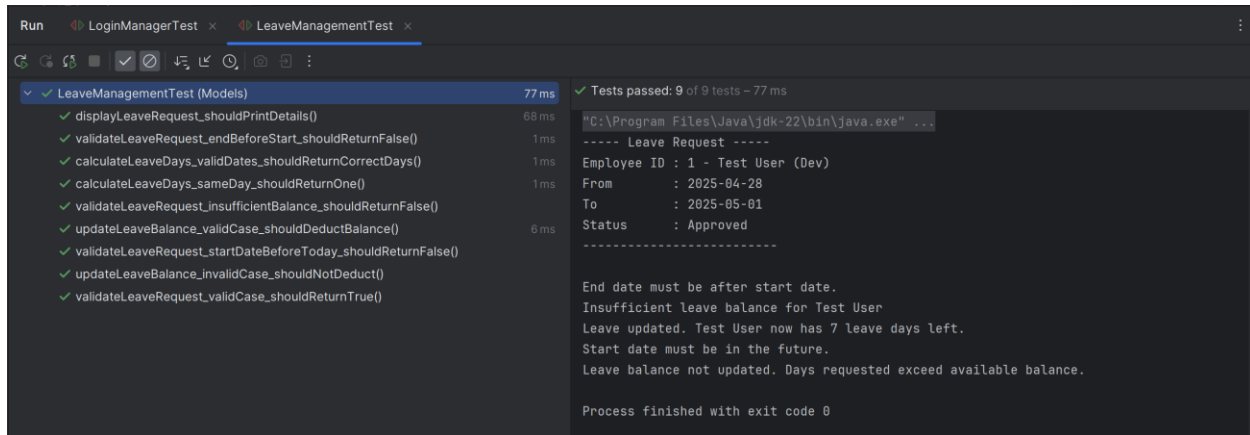
Test Setup:

```

@BeforeEach
void setUp() {
    employee = new Employee( employeeId: 1, name: "Test User", department: "IT", position: "Dev",
    salary: 5000.0, remainingLeaveDays: 10, performanceRating: 90.0, password: "pass");
}

```

Test Results:



```
Run LoginManagerTest x LeaveManagementTest x
✓ LeaveManagementTest (Models) 77 ms
  ✓ displayLeaveRequest_shouldPrintDetails() 68 ms
  ✓ validateLeaveRequest_endBeforeStart_shouldReturnFalse() 1 ms
  ✓ calculateLeaveDays_validDates_shouldReturnCorrectDays() 1 ms
  ✓ calculateLeaveDays_sameDay_shouldReturnOne() 1 ms
  ✓ validateLeaveRequest_insufficientBalance_shouldReturnFalse() 1 ms
  ✓ updateLeaveBalance_validCase_shouldDeductBalance() 6 ms
  ✓ validateLeaveRequest_startDateBeforeToday_shouldReturnFalse()
  ✓ updateLeaveBalance_invalidCase_shouldNotDeduct()
  ✓ validateLeaveRequest_validCase_shouldReturnTrue()

✓ Tests passed: 9 of 9 tests - 77 ms

"C:\Program Files\Java\jdk-22\bin\java.exe" ...
----- Leave Request -----
Employee ID : 1 - Test User (Dev)
From       : 2025-04-28
To         : 2025-05-01
Status     : Approved
-----

End date must be after start date.
Insufficient leave balance for Test User
Leave updated. Test User now has 7 leave days left.
Start date must be in the future.
Leave balance not updated. Days requested exceed available balance.

Process finished with exit code 0
```

Test Cases:

- LeaveManagementtest:

1. validateLeaveRequestValidCase:

- **Description:** Ensures valid leave requests are accepted.
- **Test:** validateLeaveRequest(employee, leave) with leave from tomorrow to 3 days later.
- **Assertions:** Returns true.

2. validateLeaveRequest_startDateBeforeToday:

- **Description:** Verifies past start dates are rejected.
- **Test:** validateLeaveRequest(employee, leave) with start date yesterday.
- **Assertions:** Returns false.

3. validateLeaveRequest_invalidstartorenddates:

- **Description:** Verifies invalid dates ranges are rejected.
- **Test:** validateLeaveRequest(employee, leave) with end date before start date / start date before end dates.
- **Assertions:** Returns false.

4. validateLeaveRequest_insufficientBalance:

- **Description:** Verifies insufficient leave balance is rejected.
- **Test:** validateLeaveRequest(employee, leave) with 15-day leave (exceeds 10 days).
- **Assertions:** Returns false.

5. updateLeaveBalance_validCase:

- **Description:** Ensures valid leave deducts balance.
- **Test:** updateLeaveBalance(leave, employee) with 3-day leave.
- **Assertions:** employee.getRemainingLeaveDays() == 7.

6. updateLeaveBalance_invalidCase:

- **Description:** Verifies invalid leave does not affect balance.
- **Test:** updateLeaveBalance(leave, employee) with 15-day leave.
- **Assertions:** employee.getRemainingLeaveDays() == 10.

7. calculateLeaveDays_validCase:

- **Description:** Ensures correct leave duration calculation.
- **Test:** calculateLeaveDays(2025-04-25, 2025-04-30).
- **Assertions:** Returns 6.

8. calculateLeaveDays_sameDay:

- **Description:** Verifies same-day leave counts as one day.
- **Test:** calculateLeaveDays(2025-04-25, 2025-04-25).
- **Assertions:** Returns 1.

9. displayLeaveRequest:

- **Description:** Ensures leave details printing runs without errors.
- **Test:** displayLeaveRequest(leave) with approved leave.
- **Assertions:** No exception thrown.


```

private Employee employee;

@BeforeEach
void setUp() { employee = new Employee( employeeId: 1, name: "Test User", department: "IT", position: "Dev", salary: 5000.0, remainingLeaveDays: 10, performance: 0.0 ); }

@Test
void validateLeaveRequest_validCase_shouldReturnTrue() {
    LeaveRequest leave = new LeaveRequest(employee, LocalDate.now().plusDays( daysToAdd: 1 ), LocalDate.now().plusDays( daysToAdd: 3 ));
    assertTrue(LeaveManagement.validateLeaveRequest(employee, leave));
}

@Test
void validateLeaveRequest_startDateBeforeToday_shouldReturnFalse() {
    LeaveRequest leave = new LeaveRequest(employee, LocalDate.now().minusDays( daysToSubtract: 1 ), LocalDate.now().plusDays( daysToAdd: 1 ));
    assertFalse(LeaveManagement.validateLeaveRequest(employee, leave));
}

@Test
void validateLeaveRequest_endBeforeStart_shouldReturnFalse() {
    LeaveRequest leave = new LeaveRequest(employee, LocalDate.now().plusDays( daysToAdd: 5 ), LocalDate.now().plusDays( daysToAdd: 3 ));
    assertFalse(LeaveManagement.validateLeaveRequest(employee, leave));
}

@Test
void validateLeaveRequest_insufficientBalance_shouldReturnFalse() {
    LeaveRequest leave = new LeaveRequest(employee, LocalDate.now().plusDays( daysToAdd: 1 ), LocalDate.now().plusDays( daysToAdd: 15 )); // 15 days
    assertFalse(LeaveManagement.validateLeaveRequest(employee, leave));
}

```

```

@Test
void updateLeaveBalance_validCase_shouldDeductBalance() {
    LeaveRequest leave = new LeaveRequest(employee, LocalDate.now().plusDays( daysToAdd: 1 ), LocalDate.now().plusDays( daysToAdd: 3 )); // 3 days
    int original = employee.getRemainingLeaveDays();

    LeaveManagement.updateLeaveBalance(leave, employee);

    assertEquals( expected: original - 3, employee.getRemainingLeaveDays());
}

@Test
void updateLeaveBalance_invalidCase_shouldNotDeduct() {
    LeaveRequest leave = new LeaveRequest(employee, LocalDate.now().plusDays( daysToAdd: 1 ), LocalDate.now().plusDays( daysToAdd: 15 )); // 15 days
    int original = employee.getRemainingLeaveDays();

    LeaveManagement.updateLeaveBalance(leave, employee);

    // Should remain unchanged
    assertEquals(original, employee.getRemainingLeaveDays());
}

@Test
void calculateLeaveDays_validDates_shouldReturnCorrectDays() {
    LocalDate start = LocalDate.of( year: 2025, month: 4, dayOfMonth: 25 );
    LocalDate end = LocalDate.of( year: 2025, month: 4, dayOfMonth: 30 );

    int days = LeaveManagement.calculateLeaveDays(start, end);
    assertEquals( expected: 6, days);
}

```

```

@Test
void calculateLeaveDays_sameDay_shouldReturnOne() {
    LocalDate date = LocalDate.of( year: 2025, month: 4, dayOfMonth: 25);
    int days = LeaveManagement.calculateLeaveDays(date, date);
    assertEquals( expected: 1, days);
}

@Test
void displayLeaveRequest_shouldPrintDetails() {
    LeaveRequest leave = new LeaveRequest(employee, LocalDate.now().plusDays( daysToAdd: 2), LocalDate.now().plusDays( daysToAdd: 5));
    leave.setApproved(true);

    // Just verify it runs without exception
    assertDoesNotThrow(() -> LeaveManagement.displayLeaveRequest(leave));
}
}

```

4.1.4 Payroll Management Test Suite:

Test Class:

- PayrollTest (model test)

Purpose: Tests payroll calculations and service operations for salary records.

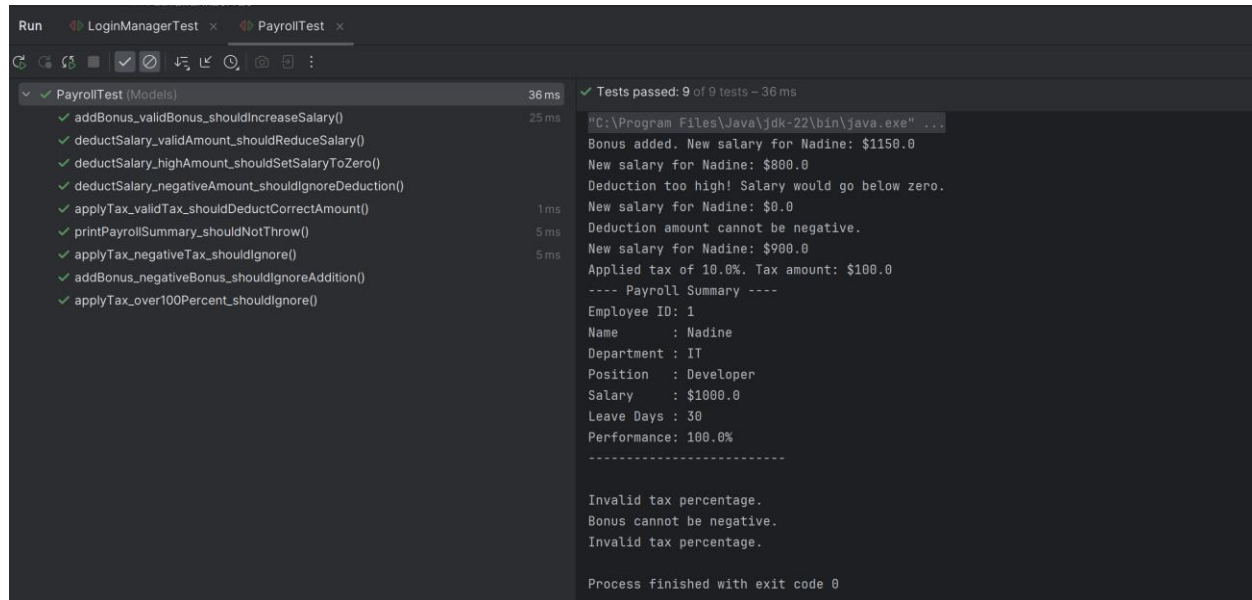
Test Setup:

```

@BeforeEach
void setUp() {
    // Initial setup for each test
    employee = new Employee( employeeId: 1, name: "Nadine", department: "IT", position: "Developer",
    salary: 1000.0, remainingLeaveDays: 30, performanceRating: 100.0, password: "password123");
}

```

Test Results:



```
Run LoginManagerTest x PayrollTest x
✓ PayrollTest (Models) 36 ms
  ✓ addBonus_validBonus_shouldIncreaseSalary() 25 ms
  ✓ deductSalary_validAmount_shouldReduceSalary()
  ✓ deductSalary_highAmount_shouldSetSalaryToZero()
  ✓ deductSalary_negativeAmount_shouldIgnoreDeduction()
  ✓ applyTax_validTax_shouldDeductCorrectAmount() 1 ms
  ✓ printPayrollSummary_shouldNotThrow() 5 ms
  ✓ applyTax_negativeTax_shouldIgnore() 5 ms
  ✓ addBonus_negativeBonus_shouldIgnoreAddition()
  ✓ applyTax_over100Percent_shouldIgnore()

✓ Tests passed: 9 of 9 tests - 36 ms

"C:\Program Files\Java\jdk-22\bin\java.exe" ...
Bonus added. New salary for Nadine: $1150.0
New salary for Nadine: $800.0
Deduction too high! Salary would go below zero.
New salary for Nadine: $0.0
Deduction amount cannot be negative.
New salary for Nadine: $900.0
Applied tax of 10.0%. Tax amount: $100.0
---- Payroll Summary ----
Employee ID: 1
Name      : Nadine
Department : IT
Position  : Developer
Salary    : $1000.0
Leave Days : 30
Performance: 100.0%
-----

Invalid tax percentage.
Bonus cannot be negative.
Invalid tax percentage.

Process finished with exit code 0
```

Test Cases:

- PayrollTest:

1. testDeductSalaryValid:

- **Description:** Deducts valid amount from salary.
- **Test:** deductSalary(employee, 200), salary 1000.0.
- **Assertions:** employee.getSalary() == 800.0.

2. testDeductSalaryHighAmount:

- **Description:** Verifies deduction caps salary at 0.
- **Test:** deductSalary(employee, 1200)
- **Assertions:** employee.getSalary() == 0.0.

3. testDeductSalaryNegative:

- **Description:** Handles negative deduction.
- **Test:** deductSalary(employee, -50).
- **Assertions:** Salary unchanged, logs error.

4. testAddBonusValid:

- **Description:** Adds valid bonus increases salary.
- **Test:** addBonus(employee, 300).
- **Assertions:** employee.getSalary() == 1300.0.

5. testApplyTaxValid:

- **Description:** Applies valid tax rate.
- **Test:** applyTax(employee, 10).
- **Assertions:** employee.getSalary() == 900.0.

6. testAddBonusNegative:

- **Description:** Handles negative bonus amount.
- **Test:** addBonus(employee, -50).
- **Assertions:** employee.getSalary() == 1000.0 (salary unchanged).

7. testApplyTaxOver100Percent:

- **Description:** Ignores tax rate exceeding 100%.
- **Test:** applyTax(employee, 150).
- **Assertions:** employee.getSalary() == 1000.0 (salary unchanged).

8. testApplyTaxNegative:

- **Description:** Ignores negative tax rate.
- **Test:** applyTax(employee, -5).
- **Assertions:** employee.getSalary() == 1000.0 (salary unchanged).

9. testPrintPayrollSummary:

- **Description:** Verifies payroll summary prints without errors.
- **Test:** printPayrollSummary(employee).
- **Assertions:** No exceptions thrown.

```

@Test
void applyTax_validTax_shouldDeductCorrectAmount() {
    Payroll.applyTax(employee, taxPercent: 10); // 10% of 1000 = 100
    assertEquals( expected: 900.0, employee.getSalary(), delta: 0.01);
}

@Test
void applyTax_over100Percent_shouldIgnore() {
    Payroll.applyTax(employee, taxPercent: 150);
    assertEquals( expected: 1000.0, employee.getSalary(), delta: 0.01);
}

@Test
void applyTax_negativeTax_shouldIgnore() {
    Payroll.applyTax(employee, taxPercent: -5);
    assertEquals( expected: 1000.0, employee.getSalary(), delta: 0.01);
}

@Test
void printPayrollSummary_shouldNotThrow() { assertDoesNotThrow(() -> Payroll.printPayrollSummary(employee)); }
}

```

```

@Test
void deductSalary_validAmount_shouldReduceSalary() {
    Payroll.deductSalary(employee, amount: 200);
    assertEquals( expected: 800.0, employee.getSalary(), delta: 0.01);
}

@Test
void deductSalary_highAmount_shouldSetSalaryToZero() {
    Payroll.deductSalary(employee, amount: 1200);
    assertEquals( expected: 0.0, employee.getSalary(), delta: 0.01);
}

@Test
void deductSalary_negativeAmount_shouldIgnoreDeduction() {
    Payroll.deductSalary(employee, amount: -100);
    assertEquals( expected: 1000.0, employee.getSalary(), delta: 0.01);
}

@Test
void addBonus_validBonus_shouldIncreaseSalary() {
    Payroll.addBonus(employee, bonus: 150);
    assertEquals( expected: 1150.0, employee.getSalary(), delta: 0.01);
}

@Test
void addBonus_negativeBonus_shouldIgnoreAddition() {
    Payroll.addBonus(employee, bonus: -50);
    assertEquals( expected: 1000.0, employee.getSalary(), delta: 0.01);
}
}

```

4.1.5 Performance Evaluation Test Suite:

Test Class:

- PerformanceEvaluationTest (model test)

Purpose: Tests performance rating calculations and service operations.

Test Setup:

```
@BeforeEach
void setUp() {
    employee = new Employee( employeeId: 1, name: "Test User", department: "IT", position: "Dev",
                             salary: 5000.0, remainingLeaveDays: 30, performanceRating: 100.0, password: "test123");
}
```

Test Results:

Run LoginManagerTest x PerformanceEvaluationTest x

✓ PerformanceEvaluationTest (Models) 35 ms

- ✓ displayPerformance_shouldNotThrow() 26 ms
- ✓ penalizePerformance_withinLimit_shouldDecrease() 6 ms
- ✓ setPerformanceManually_invalidNegativeValue_shouldIgnore()
- ✓ calculatePerformanceRate_noLeaveUsed_shouldStay100() 1 ms
- ✓ penalizePerformance_belowZero_shouldClamp()
- ✓ boostPerformance_above100_shouldClamp() 2 ms
- ✓ setPerformanceManually_above100_shouldIgnore()
- ✓ setPerformanceManually_validValue_shouldUpdate()
- ✓ calculatePerformanceRate_used15Days_shouldBe90()
- ✓ boostPerformance_withinLimit_shouldIncrease()
- ✓ calculatePerformanceRate_usedOver30_shouldNotBeNegative()
- ✓ calculatePerformanceRate_used25Days_shouldBe80()

✓ Tests passed: 12 of 12 tests – 35 ms

"C:\Program Files\Java\jdk-22\bin\java.exe" ...

---- Performance Report ----

Employee ID : 1

Name : Test User

Rating : 100.0%

Penalized performance for Test User to 70.0%

Invalid performance rating. Must be between 0 and 100.

Performance updated for Test User: 100.0%

Penalized performance for Test User to 0.0%

Boosted performance for Test User to 100.0%

Invalid performance rating. Must be between 0 and 100.

Manual performance set for Test User: 75.5%

Performance updated for Test User: 90.0%

Boosted performance for Test User to 90.0%

Performance updated for Test User: 80.0%

Performance updated for Test User: 80.0%

Process finished with exit code 0

Test Cases:

- PerformanceEvaluationTest:

1. testCalculatePerformanceRateNoLeaveUsed:

- **Description:** Verifies performance rating stays at 100 when no leave days are used.
- **Test:** calculatePerformanceRate(employee, 30).
- **Assertions:** employee.getPerformanceRating() == 100.0.

2. testCalculatePerformanceRateUsed15Days:

- **Description:** Checks performance rating is set to 90 when 15 leave days are used.
- **Test:** calculatePerformanceRate(employee, 15).
- **Assertions:** employee.getPerformanceRating() == 90.0.

3. **testCalculatePerformanceRateUsed25Days:**

- **Description:** Checks performance rating is set to 80 when 25 leave days are used.
- **Test:** calculatePerformanceRate(employee, 5).
- **Assertions:** employee.getPerformanceRating() == 80.0.

4. **testCalculatePerformanceRateUsedOver30:**

- **Description:** Ensures performance rating clamps to 80 for excessive leave days.
- **Test:** calculatePerformanceRate(employee, -5).
- **Assertions:** employee.getPerformanceRating() == 80.0.

5. **testSetPerformanceManuallyValidValue:**

- **Description:** Verifies manual setting of a valid performance rating.
- **Test:** setPerformanceManually(employee, 75.5).
- **Assertions:** employee.getPerformanceRating() == 75.5.

6. **testSetPerformanceManuallyInvalidNegativeValue:**

- **Description:** Checks that negative performance rating is ignored.
- **Test:** setPerformanceManually(employee, -10).
- **Assertions:** employee.getPerformanceRating() == 100.0 (unchanged).

7. **testSetPerformanceManuallyAbove100:**

- **Description:** Checks that performance rating above 100 is ignored.
- **Test:** setPerformanceManually(employee, 120).
- **Assertions:** employee.getPerformanceRating() == 100.0 (unchanged).

8. **testBoostPerformanceWithinLimit:**

- **Description:** Verifies performance rating increases within 100 limit.
- **Test:** setPerformanceRating(80) then boostPerformance(employee, 10).
- **Assertions:** employee.getPerformanceRating() == 90.0.

9. **testBoostPerformanceAbove100:**

- **Description:** Ensures performance rating clamps to 100 when boosted beyond.
- **Test:** setPerformanceRating(95) then boostPerformance(employee, 20).
- **Assertions:** employee.getPerformanceRating() == 100.0.

10. testPenalizePerformanceWithinLimit:

- **Description:** Verifies performance rating decreases within 0 limit.
- **Test:** setPerformanceRating(90) then penalizePerformance(employee, 20).
- **Assertions:** employee.getPerformanceRating() == 70.0.

11. testPenalizePerformanceBelowZero:

- **Description:** Ensures performance rating clamps to 0 when penalized below.
- **Test:** setPerformanceRating(10) then penalizePerformance(employee, 50).
- **Assertions:** employee.getPerformanceRating() == 0.0.

12. testDisplayPerformance:

- **Description:** Ensures performance display runs without errors.
- **Test:** displayPerformance(employee).
- **Assertions:** No exceptions thrown.

```
@Test
void calculatePerformanceRate_noLeaveUsed_shouldStay100() {
    PerformanceEvaluation.calculatePerformanceRate(employee, remainingLeaveDays: 30); // Used 0 days
    assertEquals( expected: 100.0, employee.getPerformanceRating(), delta: 0.01);
}

@Test
void calculatePerformanceRate_used15Days_shouldBe90() {
    PerformanceEvaluation.calculatePerformanceRate(employee, remainingLeaveDays: 15); // Used 15 days
    assertEquals( expected: 90.0, employee.getPerformanceRating(), delta: 0.01);
}

@Test
void calculatePerformanceRate_used25Days_shouldBe80() {
    PerformanceEvaluation.calculatePerformanceRate(employee, remainingLeaveDays: 5); // Used 25 days
    assertEquals( expected: 80.0, employee.getPerformanceRating(), delta: 0.01);
}

@Test
void calculatePerformanceRate_usedOver30_shouldNotBeNegative() {
    PerformanceEvaluation.calculatePerformanceRate(employee, remainingLeaveDays: -5); // Used 35 days (invalid input)
    assertEquals( expected: 80.0, employee.getPerformanceRating(), delta: 0.01); // Still clamps to minimum 80 as per your l
}

@Test
void setPerformanceManually_validValue_shouldUpdate() {
    PerformanceEvaluation.setPerformanceManually(employee, rating: 75.5);
    assertEquals( expected: 75.5, employee.getPerformanceRating(), delta: 0.01);
}
```



```

@Test
void setPerformanceManually_invalidNegativeValue_shouldIgnore() {
    PerformanceEvaluation.setPerformanceManually(employee, rating: -10);
    assertEquals( expected: 100.0, employee.getPerformanceRating(), delta: 0.01); // Unchanged
}

```

```

@Test
void setPerformanceManually_above100_shouldIgnore() {
    PerformanceEvaluation.setPerformanceManually(employee, rating: 120);
    assertEquals( expected: 100.0, employee.getPerformanceRating(), delta: 0.01); // Unchanged
}

```

```

@Test
void boostPerformance_withinLimit_shouldIncrease() {
    employee.setPerformanceRating(80);
    PerformanceEvaluation.boostPerformance(employee, boostAmount: 10);
    assertEquals( expected: 90.0, employee.getPerformanceRating(), delta: 0.01);
}

```

```

@Test
void boostPerformance_above100_shouldClamp() {
    employee.setPerformanceRating(95);
    PerformanceEvaluation.boostPerformance(employee, boostAmount: 20);
    assertEquals( expected: 100.0, employee.getPerformanceRating(), delta: 0.01);
}

```

```

@Test
void penalizePerformance_withinLimit_shouldDecrease() {
    employee.setPerformanceRating(90);
    PerformanceEvaluation.penalizePerformance(employee, penaltyAmount: 20);
    assertEquals( expected: 70.0, employee.getPerformanceRating(), delta: 0.01);
}

```

```

@Test
void penalizePerformance_belowZero_shouldClamp() {
    employee.setPerformanceRating(10);
    PerformanceEvaluation.penalizePerformance(employee, penaltyAmount: 50);
    assertEquals( expected: 0.0, employee.getPerformanceRating(), delta: 0.01);
}

```

```

@Test
void displayPerformance_shouldNotThrow() {
    assertDoesNotThrow(() -> PerformanceEvaluation.displayPerformance(employee));
}

```

4.1.6 Authentication Test Suite:

Test Class:

- LoginManagerTest

Purpose: Validates user authentication logic.

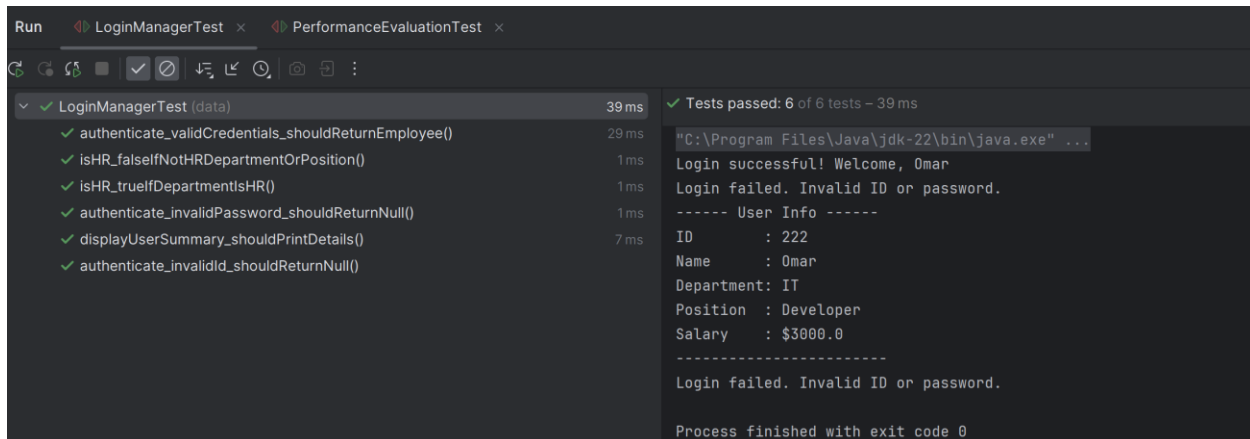
Test Setup:

```
@BeforeEach
void setUp() {
    employeeList = new ArrayList<>();

    hrEmployee = new Employee( employeeId: 111, name: "Yasmine", department: "HR", position: "Manager",
                               salary: 5000.0, remainingLeaveDays: 30, performanceRating: 100.0, password: "passHR");
    nonHrEmployee = new Employee( employeeId: 222, name: "Omar", department: "IT", position: "Developer",
                                   salary: 3000.0, remainingLeaveDays: 30, performanceRating: 100.0, password: "passIT");

    employeeList.add(hrEmployee);
    employeeList.add(nonHrEmployee);
}
```

Test Results:



Run LoginManagerTest x PerformanceEvaluationTest x

✓ LoginManagerTest (data) 39 ms

- ✓ authenticate_validCredentials_shouldReturnEmployee() 29 ms
- ✓ isHR_falseIfNotHRDepartmentOrPosition() 1 ms
- ✓ isHR_trueIfDepartmentIsHR() 1 ms
- ✓ authenticate_invalidPassword_shouldReturnNull() 1 ms
- ✓ displayUserSummary_shouldPrintDetails() 7 ms
- ✓ authenticate_invalidId_shouldReturnNull()

✓ Tests passed: 6 of 6 tests – 39 ms

"C:\Program Files\Java\jdk-22\bin\java.exe" ...

Login successful! Welcome, Omar

Login failed. Invalid ID or password.

----- User Info -----

ID : 222

Name : Omar

Department: IT

Position : Developer

Salary : \$3000.0

Login failed. Invalid ID or password.

Process finished with exit code 0

Test Cases:

1. testAuthenticateValidCredentials:

- **Description:** Verifies successful authentication with valid ID and password.
- **Test:** authenticate(employeeList, 222, "passIT").
- **Assertions:** Returns non-null Employee object with name "Omar".

2. **testAuthenticateInvalidPassword:**

- **Description:** Checks authentication failure with incorrect password.
- **Test:** authenticate(employeeList, 222, "wrongPass").
- **Assertions:** Returns null.

3. **testAuthenticateInvalidId:**

- **Description:** Checks authentication failure with non-existent employee ID.
- **Test:** authenticate(employeeList, 999, "passIT").
- **Assertions:** Returns null.

4. **testIsHRTrueIfDepartmentIsHR:**

- **Description:** Verifies isHR returns true for HR department employee.
- **Test:** isHR(hrEmployee).
- **Assertions:** Returns true.

5. **testIsHRFalseIfNotHRDepartmentOrPosition:**

- **Description:** Verifies isHR returns false for non-HR department employee.
- **Test:** isHR(nonHrEmployee).
- **Assertions:** Returns false.

6. **testDisplayUserSummary:**

- **Description:** Ensures user summary displays without errors.
- **Test:** displayUserSummary(nonHrEmployee).
- **Assertions:** No exceptions thrown.

```

@Test
void authenticate_validCredentials_shouldReturnEmployee() {
    Employee result = LoginManager.authenticate(employeeList, id: 222, password: "passIT");
    assertNotNull(result);
    assertEquals( expected: "Omar", result.getName());
}

@Test
void authenticate_invalidPassword_shouldReturnNull() {
    Employee result = LoginManager.authenticate(employeeList, id: 222, password: "wrongPass");
    assertNull(result);
}

@Test
void authenticate_invalidId_shouldReturnNull() {
    Employee result = LoginManager.authenticate(employeeList, id: 999, password: "passIT");
    assertNull(result);
}

@Test
void isHR_trueIfDepartmentIsHR() { assertTrue(LoginManager.isHR(hrEmployee)); }

@Test
void isHR_falseIfNotHRDepartmentOrPosition() { assertFalse(LoginManager.isHR(nonHrEmployee)); }

@Test
void displayUserSummary_shouldPrintDetails() {
    assertDoesNotThrow(() -> LoginManager.displayUserSummary(nonHrEmployee));
}
}

```

4.1.7 File Handler Test Suite:

Test Class:

- FileHandlerTest

Purpose: Validates file I/O operations for data persistence.

Test Setup:

```
@BeforeEach
void setUp() {
    sampleEmployees = new ArrayList<>();
    Employee emp1 = new Employee( employeeId: 1, name: "Alice", department: "IT", position: "Developer",
    salary: 5000.0, remainingLeaveDays: 30, performanceRating: 100.0, password: "pass123");
    Employee emp2 = new Employee( employeeId: 2, name: "Bob", department: "HR", position: "Manager",
    salary: 6000.0, remainingLeaveDays: 25, performanceRating: 90.0, password: "pass456");
    sampleEmployees.add(emp1);
    sampleEmployees.add(emp2);

    sampleLeaves = new ArrayList<>();
    sampleLeaves.add(new LeaveRequest(emp1, LocalDate.of( year: 2025, month: 5, dayOfMonth: 10),
    LocalDate.of( year: 2025, month: 5, dayOfMonth: 15)));
    LeaveRequest lr = new LeaveRequest(emp2, LocalDate.of( year: 2025, month: 6, dayOfMonth: 1),
    LocalDate.of( year: 2025, month: 6, dayOfMonth: 3));
    lr.setApproved(true);
    sampleLeaves.add(lr);
}
```

Test Cases:

1. testSaveEmployees:

- **Description:** Verifies that employee data is saved to a file.
- **Test:** saveEmployees(sampleEmployees).
- **Assertions:** employees.txt exists and is not empty.

2. testLoadEmployees:

- **Description:** Checks that saved employee data is correctly loaded.
- **Test:** saveEmployees(sampleEmployees) then loadEmployees().
- **Assertions:** Loaded list has 2 employees with names "Alice" and "Bob".

3. testSaveLeaves:

- **Description:** Verifies that leave request data is saved to a file.
- **Test:** saveLeaves(sampleLeaves).
- **Assertions:** leaves.txt exists and is not empty.

4. testLoadLeaves:

- **Description:** Checks that saved leave requests are correctly loaded.
- **Test:** saveEmployees(sampleEmployees), saveLeaves(sampleLeaves), then loadLeaves(loadedEmployees).
- **Assertions:** Loaded list has 2 leave requests, with correct start date (2025-05-10) and approval status (true for second request).

```

@Test
void saveEmployees() {
    FileHandler.saveEmployees(sampleEmployees);
    File file = new File(EMPLOYEE_FILE);
    assertTrue( condition: file.exists() && file.length() > 0, message: "Employee file should exist and not be empty");
}

@Test
void loadEmployees() {
    FileHandler.saveEmployees(sampleEmployees);
    ArrayList<Employee> loaded = FileHandler.loadEmployees();

    assertEquals( expected: 2, loaded.size());
    assertEquals( expected: "Alice", loaded.get(0).getName());
    assertEquals( expected: "Bob", loaded.get(1).getName());
}

```

```

@Test
void saveLeaves() {
    FileHandler.saveLeaves(sampleLeaves);
    File file = new File(LEAVE_FILE);
    assertTrue( condition: file.exists() && file.length() > 0, message: "Leave file should exist and not be empty");
}

@Test
void loadLeaves() {
    FileHandler.saveEmployees(sampleEmployees);
    FileHandler.saveLeaves(sampleLeaves);

    ArrayList<Employee> loadedEmployees = FileHandler.loadEmployees();
    ArrayList<LeaveRequest> loadedLeaves = FileHandler.loadLeaves(loadedEmployees);

    assertEquals( expected: 2, loadedLeaves.size());
    assertEquals(LocalDate.of( year: 2025, month: 5, dayOfMonth: 10), loadedLeaves.get(0).getStartDate());
    assertTrue(loadedLeaves.get(1).isApproved());
}

```

5. White Box Testing:

White-box testing was performed on three key classes from the Models package (**Payroll**, **LeaveManagement**, **PerformanceEvaluation**), focusing on internal logic, code paths, and branch coverage. The test files (**PayrollTestW.java**, **LeaveManagementTestW.java**, **PerformanceEvaluationTestW.java**) target 100% line and branch coverage, as verified by JaCoCo reports referenced in workspace.xml. Tests include statement, branch, and path coverage, with assertions on internal state and console output (captured via **ByteArrayOutputStream**). The tests also identify potential improvements, such as adding validation for negative salaries or null inputs.

Below is a detailed list of all white-box test cases, organized by class, with inputs, expected outputs, code paths tested, and purposes.

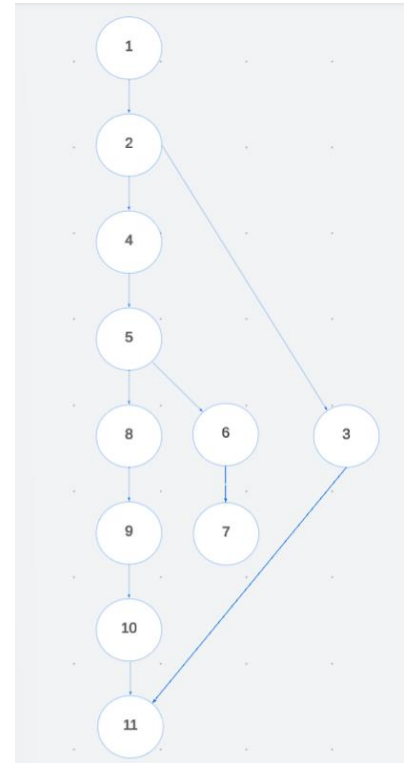
Element ^	Class, %	Method, %	Line, %	Branch, %
▼  Models	80% (4/5)	89% (34/38)	75% (88/117)	67% (23/34)
 Employee	100% (1/1)	100% (19/19)	100% (27/27)	100% (0/0)
 LeaveManagement	0% (0/1)	0% (0/4)	0% (0/29)	0% (0/10)
 LeaveRequest	100% (1/1)	100% (6/6)	100% (10/10)	100% (0/0)
 Payroll	100% (1/1)	100% (4/4)	100% (26/26)	100% (10/10)
 PerformanceEvaluation	100% (1/1)	100% (5/5)	100% (25/25)	92% (13/14)

5.1 Payroll Class:

5.1.1 Function 1: deductSalary(employee, amount)

1. **Start:** Begin execution of deductSalary(employee, amount).
2. **Check if amount < 0:**
 - 2.1 **TRUE** → Proceed to step 3.
 - 2.2 **FALSE** → Proceed to step 4.
3. **Print error:** Print error, Go to step 11.
4. **Calculate newSalary**
5. **Check if newSalary < 0:**
 - 5.1 **TRUE** → Proceed to step 6.
 - 5.2 **FALSE** → Proceed to step 8.
6. **Set salary to 0:** Set employee.setSalary(0).
7. **Print over-deduction error**
8. **Update salary:** Set employee.setSalary(newSalary).
9. **Prepare output**
10. **Print new salary:** Output the formatted string.
11. **End:** Finish execution.

- ★ Statement Coverage = 100%
- ★ Branch Coverage = 100%

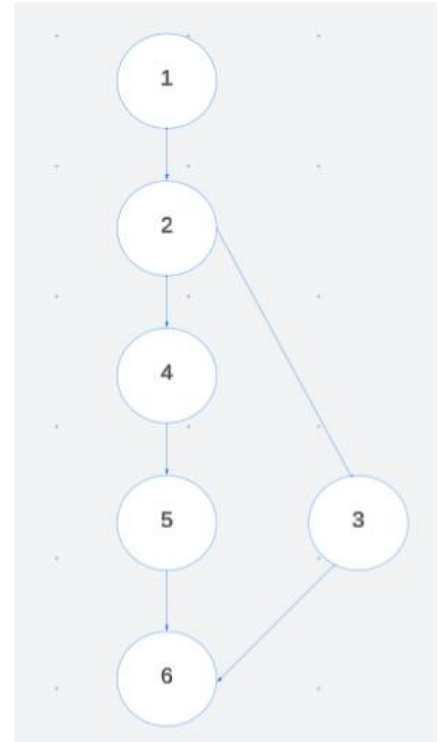


Test case ID	Goal	Covered Statements	Expected Output	Pass/Fail
TC1	Negative deduction amount	1,2,3	Salary unchanged (1000.0), Print "cannot be negative"	Pass
TC2	Valid deduction	1,2,4,5,8,9,10,11	Salary = 750.0 (after deducting 250)	Pass
TC3	Deduction causing salary < 0	1, 2, 4, 5, 6, 7, 10, 11	Salary = 0.0, Print "Salary would go below zero"	Pass

5.1.2 Function 2: addBonus(employee, bonus)

1. **Start:** Begin execution of addBonus(employee, bonus).
2. **Check if bonus < 0:**
 - 2.1 **TRUE** → Proceed to step 3.
 - 2.2 **FALSE** → Proceed to step 4.
3. **Print error:** Print "Bonus cannot be negative."
Go to step 6.
4. **Update salary**
5. **Print success message**
6. **End:** Finish execution.

- ★ Statement Coverage = 100%
- ★ Branch Coverage = 100%

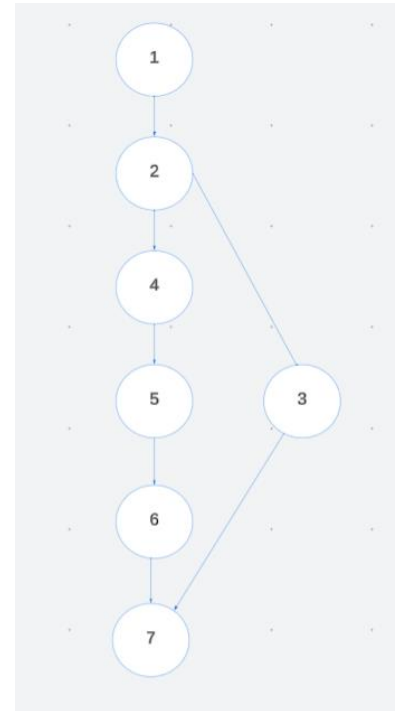


Test case ID	Goal	Covered Statements	Expected Output	Pass/Fail
TC1	Negative bonus	1,2,3	Salary unchanged (1000.0), Print "Bonus cannot be negative"	Pass
TC2	Positive bonus	1, 2, 4, 5, 6	Salary = 1200.0 (after adding 200)	Pass

5.1.3 Function 3: applyTax(employee, taxPercent)

1. **Start:** Begin execution of applyTax(employee, taxPercent).
2. **Check if taxPercent < 0 or > 100:**
 - 2.1 **TRUE** → Proceed to step 3.
 - 2.2 **FALSE** → Proceed to step 4.
3. **Print error:** Print "Invalid tax percentage."
Go to step 7.
4. **Calculate tax amount**
5. **Call deductSalary:**
6. **Print tax message**
7. **End:** Finish execution.

- ★ Statement Coverage = 100%
- ★ Branch Coverage = 100%

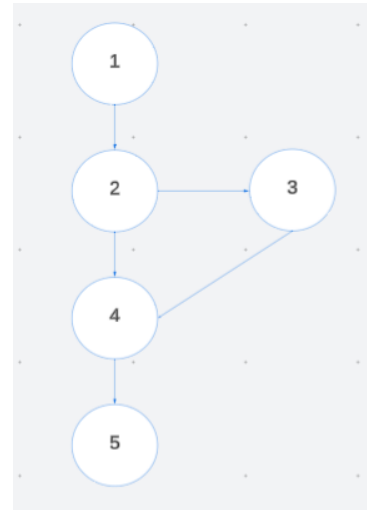


Test case ID	Goal	Covered Statements	Expected Output	Pass/Fail
TC1	Negative tax percent	1,2,3	Salary unchanged (1000.0), Print "Invalid"	Pass
TC2	Tax percent > 100	1, 2, 3	Salary unchanged (1000.0), Print "Invalid"	Pass
TC3	Valid tax percent	1, 2, 4, 5, 6, 7	Salary = 900.0 (after 10% tax), Print "Tax amount: \$100.0"	Pass

5.1.4 Function 4: printPayrollSummary(employee)

1. **Start:** Begin execution of printPayrollSummary(employee).
2. **Print header:** Output "---- Payroll Summary ----".
3. **Print employee details:** Output lines for Employee ID, Name, Department, Position, Salary, Leave Days, and Performance.
4. **Print footer:** Output "-----\n".
5. **End:** Finish execution.

- ★ Statement Coverage = 100%
- ★ Branch Coverage= 100%

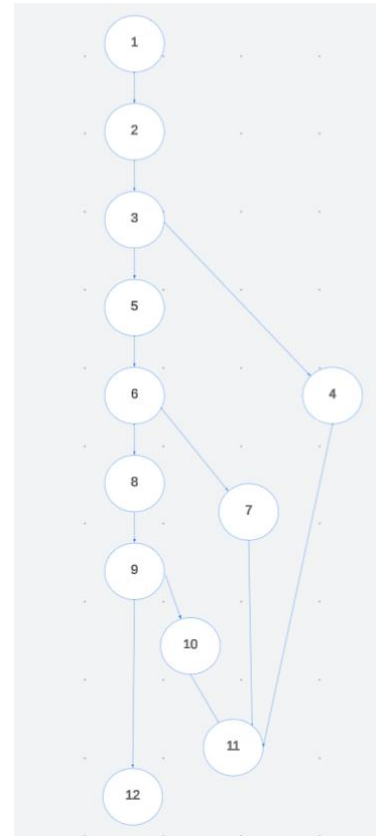


Test case ID	Goal	Covered Statements	Expected Output	Pass/Fail
TC1	Print payroll summary	1, 2, 3, 4, 5	Output contains ID, Name, Salary, Leave Days, Performance	Pass

5.2 LeaveManagement Class:

5.2.1 Function 1: validateLeaveRequest(employee, leave)

1. **Start:** Begin execution of validateLeaveRequest(employee, leave).
2. **Get today's date:** Set today = LocalDate.now().
3. **Check if start date < today:**
 - 1.1 **TRUE** → Proceed to step 4.
 - 1.2 **FALSE** → Proceed to step 5.
4. **Print error:** Print "Start date must be in the future." Go to step 11.
5. **Get start and end dates:** Retrieve start = leave.getStartDate() and end = leave.getEndDate().
6. **Check if end date <= start date:**
 - 6.1 **TRUE** → Proceed to step 7.
 - 6.2 **FALSE** → Proceed to step 8.
7. **Print error:** Print "End date must be after start date." Go to step 11.
8. **Calculate requested days:** Compute daysRequested = calculateLeaveDays(start, end).
9. **Check leave balance:**
 - 9.1 **Insufficient** → Proceed to step 10.
 - 9.2 **Sufficient** → Return true.
10. **Print error:** Print "Insufficient leave balance for [employee name]."
11. **End:** Return false.
12. **End:** return true.



- ★ Statement Coverage = 100%
- ★ Branch Coverage = 100%

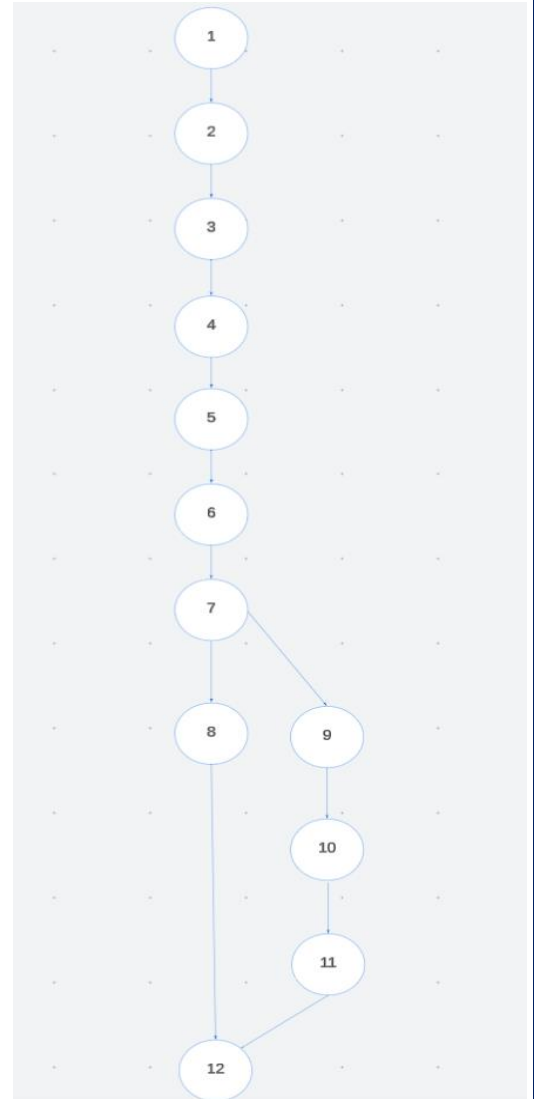
Test case ID	Goal	Covered Statements	Expected Output	Pass/Fail
TC1	Start date in past	1,2,3,4	Return false, Print "Start date"	Pass
TC2	End date <= start date	1, 2, 3, 5, 6, 7	Return false, Print "End date"	Pass
TC3	Insufficient leave balance	1, 2, 3, 5, 6, 8, 9, 10, 11	Return false, Print "Insufficient"	Pass
TC4	Valid request	1, 2, 3, 5, 6, 8, 9,12	Return true	Pass

5.2.2 Function 2: calculateLeaveDays

1. **Start:** Begin execution of calculateLeaveDays(start, end).
2. **Enter try block:** Begin try block.
3. **Get start epoch day:** Compute startEpoch = start.toEpochDay().
4. **Get end epoch day:** Compute endEpoch = end.toEpochDay().
5. **Calculate days:** Compute days = (endEpoch - startEpoch) + 1.
6. **Prepare return:** Set return value to days.
7. **Check for exception:**
 - 1.1 **Exception** → Proceed to step 9.
 - 1.2 **No exception** → Return the computed days.
2. Return computed days
3. **Catch block:** Handle DateTimeParseException.
4. **Print error:** Print "Invalid date format. Use YYYY-MM-DD."
5. **Set return value:** Set return value to -1.
6. **End:** Finish execution and return the result.

★ Statement Coverage = 100%

★ Branch Coverage= 100%

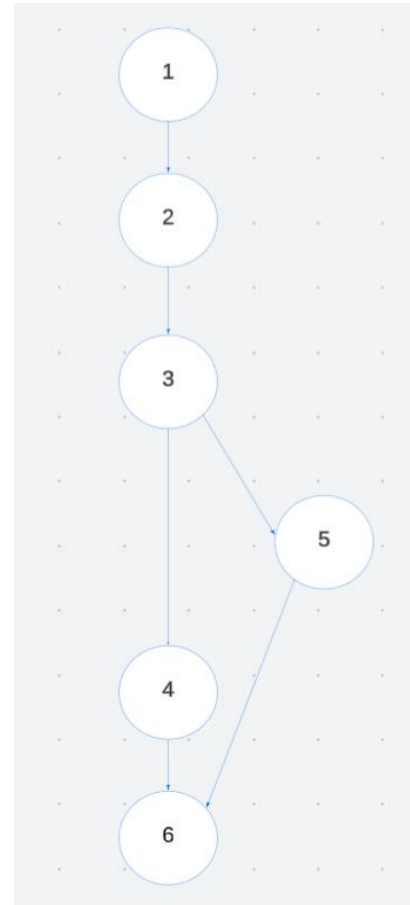


Test case ID	Goal	Covered Statements	Expected Output	Pass/Fail
TC1	One-day leave	1, 2, 3, 4, 5, 6, 7,8,12	Return 1	Pass
TC2	Multi-day leave	1, 2, 3, 4, 5, 6, 7,8,12	Return 10	Pass
TC3	Invalid date (exception)	1, 2, 3, 4, 5,6,7, 8, 9, 10, 11	Return -1, Print "Invalid date format"	Pass

5.2.3 Function 3: updateLeaveBalance

1. **Start:** Begin execution of updateLeaveBalance(leave, employee).
2. **Calculate days taken**
3. **Check if daysTaken <= employee.getRemainingLeaveDays():**
 - 1.1 **TRUE** → Proceed to step 4.
 - 1.2 **FALSE** → Proceed to step 5.
4. **Update balance:** Go to step 6.
5. **Print error:** Print "Leave balance not updated. Days requested exceed available balance."
6. **End:** Finish execution.

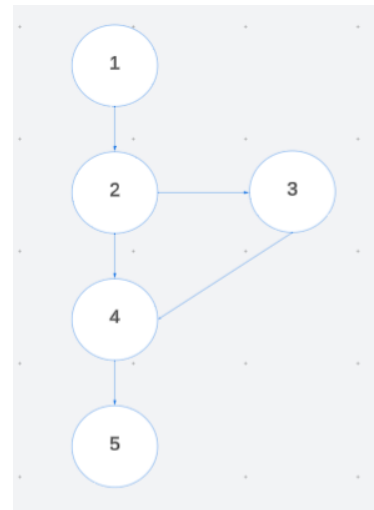
- ★ Statement Coverage = 100%
- ★ Branch Coverage = 100%



Test case ID	Goal	Covered Statements	Expected Output	Pass/Fail
TC1	Sufficient balance	1, 2, 3, 4, 6	Balance = 27, Print "now has 27"	Pass
TC2	Insufficient balance	1, 2, 3, 5, 6	Balance unchanged (2), Print "balance not updated"	Pass

5.2.4 Function 4: displayLeaveRequest

1. **Start:** Begin execution of displayLeaveRequest(leave).
2. **Print header:** Output "----- Leave Request -----".
3. **Print details:** Output lines for Employee ID, From, To, and Status.
4. **Print footer:** Output "-----\n".
5. **End:** Finish execution.



- ★ Statement Coverage = 100%
- ★ Branch Coverage = 100%

Test case ID	Goal	Covered Statements	Expected Output	Pass/Fail
TC1	Print leave request summary	1, 2, 3, 4, 5	Output contains Employee ID, From, To, Status	Pass

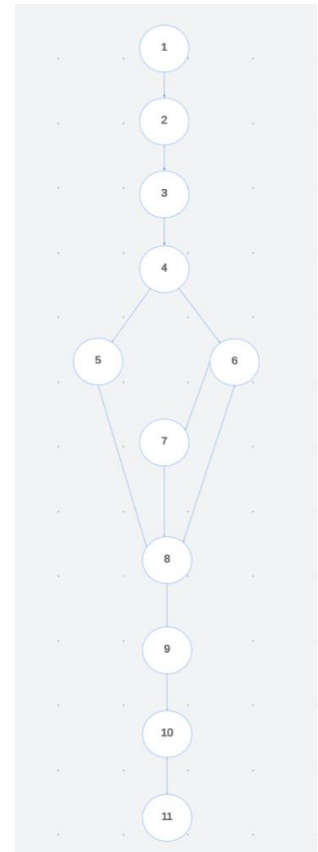
5.3 PerformanceEvaluation Class:

5.3.1 Function 1: calculatePerformanceRate

1. **Start:** Begin execution of calculatePerformanceRate(employee, remainingLeaveDays).
2. **Calculate leave used**
3. **Initialize performance:** Set performance = 100.0.
4. **Check if totalLeaveDaysUsed > 10 && totalLeaveDaysUsed <= 20:**
 - 1.1 **TRUE** → Step 5.
 - 1.2 **FALSE** → Step 6.
5. **Reduce performance by 10:** Set performance -= 10.
6. **Check if totalLeaveDaysUsed > 20:**
 - 6.1 **TRUE** → Step 7.
 - 6.2 **FALSE** → Step 8.
7. **Reduce performance by 20:** Set performance -= 20.
8. **Ensure performance >= 0:** Call Math.max(0, performance).
9. **Update employee rating**
10. **Print update message**
11. **End:** Finish execution.

★ Statement Coverage = 100%

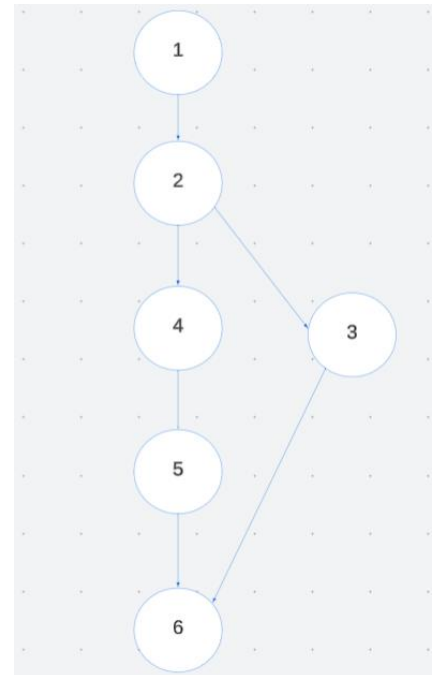
★ Branch Coverage= 100%



Test case ID	Goal	Covered Statements	Expected Output	Pass/Fail
TC1	No leave taken (0–10 days)	1, 2, 3, 4, 6, 8, 9, 10, 11	Rating = 100%	Pass
TC2	Moderate leave (11–20 days)	1, 2, 3, 4, 5, 8, 9, 10, 11	Rating = 90%	Pass
TC3	Heavy leave (21+ days)	1, 2, 3, 4, 5, 6, 7, 8, 9, 10, 11	Rating = 80%	Pass

5.3.2 Function 2: setPerformanceManually

1. **Start:** Begin execution of setPerformanceManually(employee, rating).
2. **Check if rating < 0 || rating > 100:**
 - 2.1 **TRUE** → Step 3.
 - 2.2 **FALSE** → Step 4.
3. **Print error and return:** Print "Invalid performance rating. Must be between 0 and 100." Go to step 6.
4. **Update rating:** Call employee.setPerformanceRating(rating).
5. **Print success message:** Output "Manual performance set for [employee.getName()]: [rating]%%".
6. **End:** Finish execution.

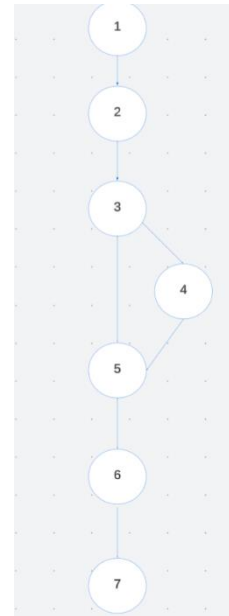


- ★ Statement Coverage = 100%
- ★ Branch Coverage = 100%

Test case ID	Goal	Covered Statements	Expected Output	Pass/Fail
TC1	Valid rating in [0, 100]	1, 2, 4, 5, 6	Rating = 55.5, Print "Manual performance set"	Pass
TC2	Invalid rating (> 100 or < 0)	1, 2, 3, 6	Rating unchanged, Print "Invalid"	Pass

5.3.3 Function 3: boostPerformance

1. **Start:** Begin execution of boostPerformance(employee, boostAmount).
2. **Calculate new rating**
3. **Check if newRating > 100:**
 - 3.1. **TRUE** → Step 4.
 - 3.2. **FALSE** → Step 5.
4. **Cap rating:** Set newRating = 100
5. **Update rating**
6. **Print boost message**
7. **End:** Finish execution.



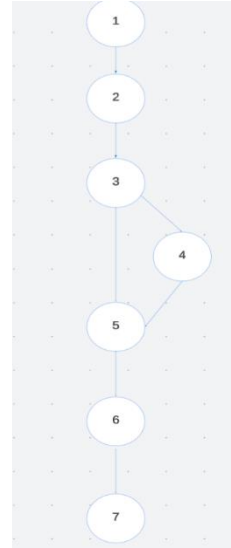
★ Statement Coverage = 100%

1. Branch Coverage= 100%

Test case ID	Goal	Covered Statements	Expected Output	Pass/Fail
TC1	Boost within range	1, 2, 3, 5, 6, 7	Rating = 90, Print "Boosted performance"	Pass
TC2	Boost capped at 100	1, 2, 3, 4, 5, 6, 7	Rating = 100, Print "Boosted performance"	Pass

5.3.4 Function 4: penalizePerformance

1. **Start:** Begin execution of penalizePerformance(employee, penaltyAmount).
2. **Calculate new rating**
3. **Check if newRating < 0:**
 - 3.1. **TRUE** → Step 4.
 - 3.2. **FALSE** → Step 5.
4. **Floor rating:** Set newRating = 0. Go to step 5.
5. **Update rating**
6. **Print penalty message**
7. **End:** Finish execution.



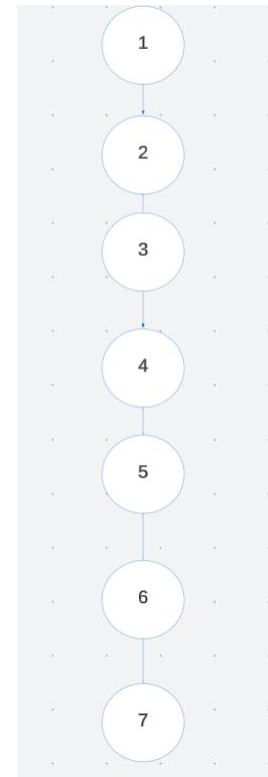
- ★ Statement Coverage = 100%
- ★ Branch Coverage = 100%

Test case ID	Goal	Covered Statements	Expected Output	Pass/Fail
TC1	Penalty within range	1, 2, 3, 5, 6, 7	Rating = 30, Print "Penalized performance"	Pass
TC2	Penalty floored at 0	1, 2, 3, 4, 5, 6, 7	Rating = 0, Print "Penalized performance"	Pass

5.3.5 Function 5: displayPerformance

2. **Start:** Begin execution of displayPerformance(employee).
3. **Print header:** Output "---- Performance Report ----".
4. **Print ID line:** Output "Employee ID : [employee.getEmployeeId()]".
5. **Print name line:** Output "Name : [employee.getName()]".
6. **Print rating line:** Output "Rating : [employee.getPerformanceRating()]".
7. **Print footer:** Output "-----\n".
8. **End:** Finish execution.

- ★ Statement Coverage = 100%
- ★ Branch Coverage = 100%



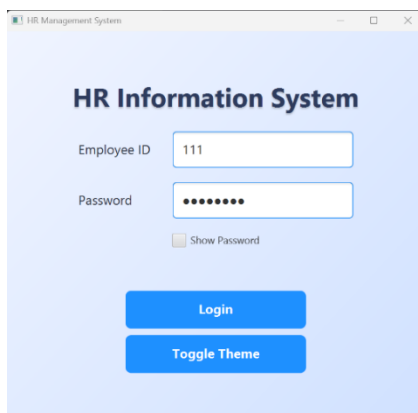
Test case ID	Goal	Covered Statements	Expected Output	Pass/Fail
TC1	Print performance summary	1, 2, 3, 4, 5, 6, 7	Output contains ID, Name, Rating	Pass

6. GUI Testing:

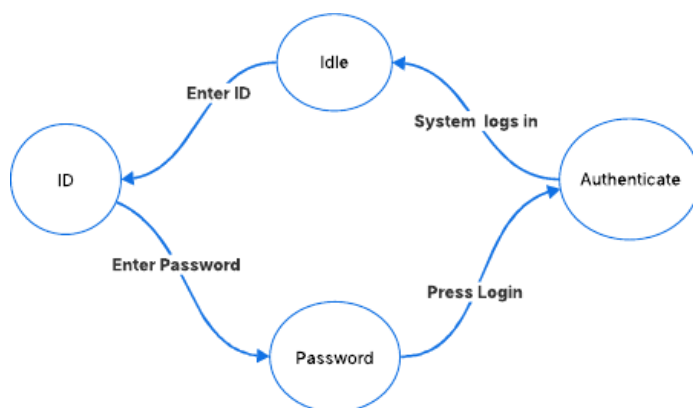
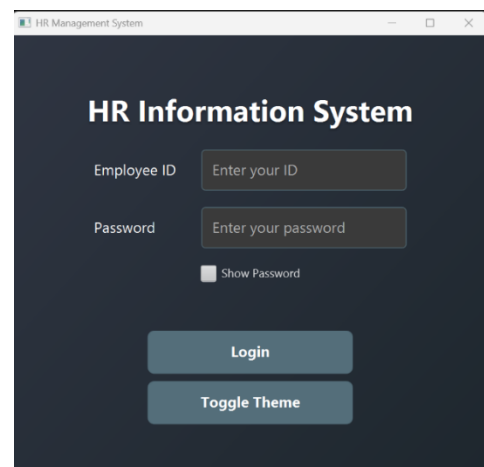
GUI testing validated the JavaFX-based user interface defined in FXML files (**login.fxml**, **hr_dashboard.fxml**, **employee_dashboard.fxml**, **add_employee.fxml**, **edit_employee.fxml**, **view_employees.fxml**, **leave_management.fxml**, **payroll.fxml**, **performance.fxml**) and controlled by corresponding controller classes (**LoginController.java**, **HRDashboardController.java**, **EmployeeDashboardController.java**, etc.). A Finite State Machine (FSM) modeled user interaction scenarios, covering state transitions such as login to dashboard, leave submission to approval, and employee data updates. Testing was conducted using manual inspection and TestFX for automation, focusing on functionality, usability, and responsiveness.

The Login page:

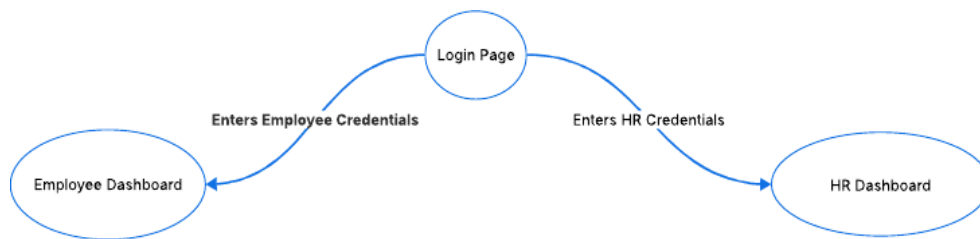
The user enters the ID and



Every toggle button switches the theme from light to dark and vice versa



Then after logging in, if the user enters the Employee credentials, then will be directed to the Employee dashboard, and if the user enters the HR credentials, then will be directed to the HR dashboard



Entering invalid data

HR Management System

HR Information System

Employee ID

Password

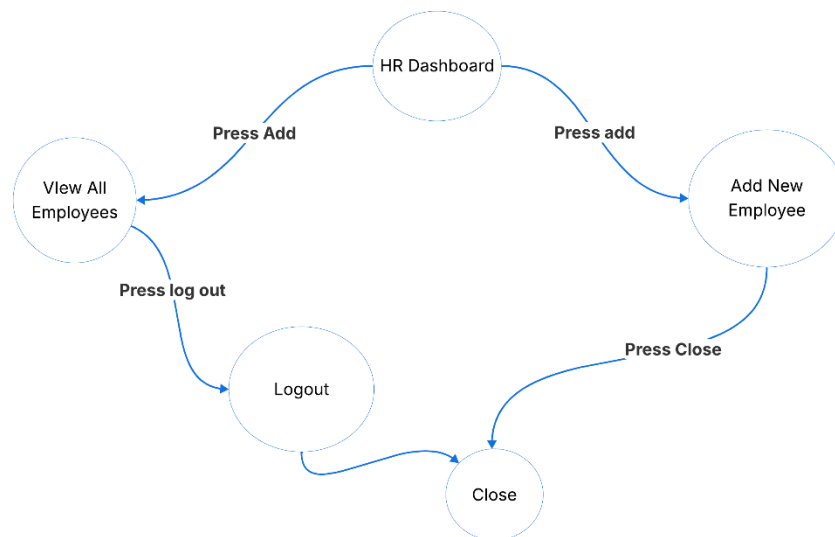
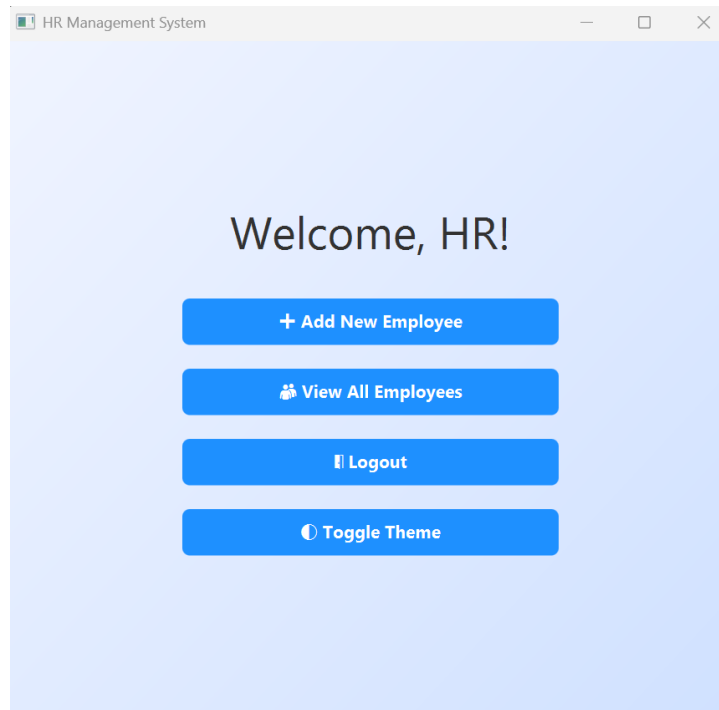
☐ Show Password

Invalid ID or password.

Login

Toggle Theme

HR Dashboard



Add New Employee

then the first button Add New Employee will then load the adding new employee page.

The image displays two side-by-side screenshots of a web application window titled "Add New Employee".

The left screenshot shows the form with pre-filled data:

- Name: Jana
- Department: Sales
- Position: Team Leader
- Salary: 20000
- Password: *****

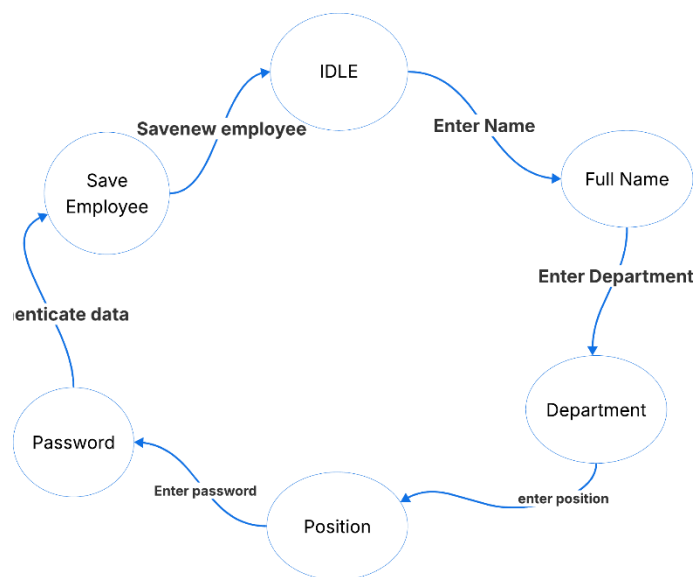
Below the form are two buttons: "Save Employee" and "Toggle Theme".

The right screenshot shows the form with placeholder text:

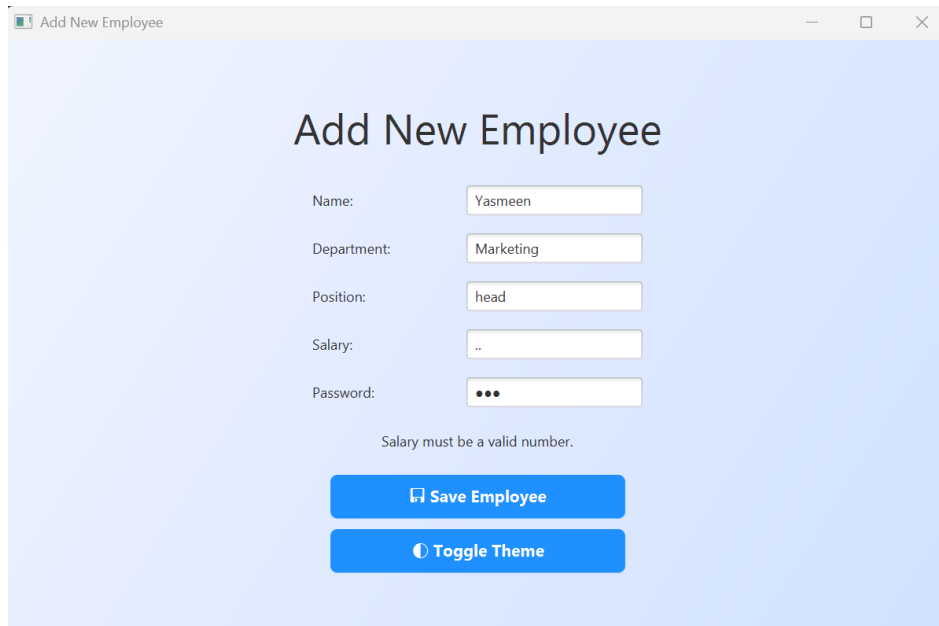
- Name: Enter full name
- Department: Enter department
- Position: Enter job title
- Salary: Enter monthly salary
- Password: Set account password

Below the form are two buttons: "Save Employee" and "Toggle Theme". Above the buttons, a message reads: "Employee saved successfully!".

After adding the new employee data then pressing save employee , it will present this message that the data is saved successfully



Entering invalid data will display the following error message :

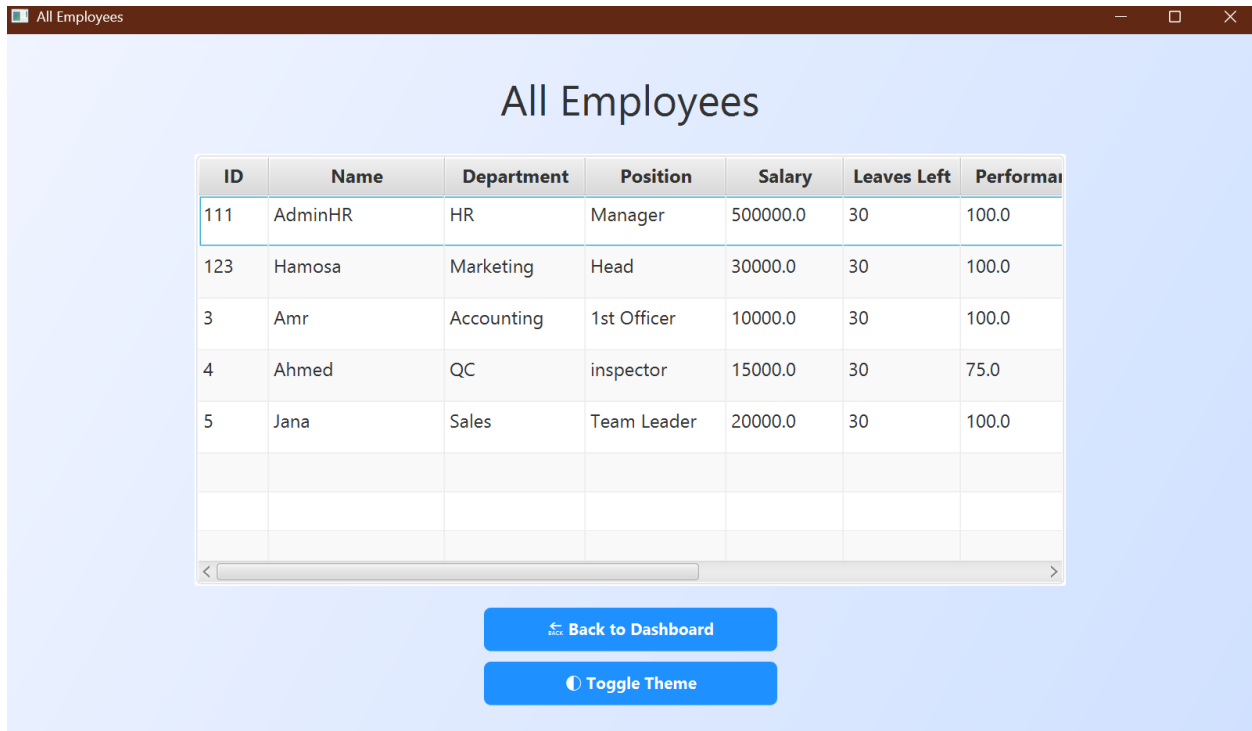


The screenshot shows a web application window titled "Add New Employee". The form contains the following fields and values:

- Name: Yasmeen
- Department: Marketing
- Position: head
- Salary: ..
- Password: ...

Below the form, an error message states: "Salary must be a valid number." At the bottom of the form, there are two buttons: "Save Employee" and "Toggle Theme".

View all Employees page:



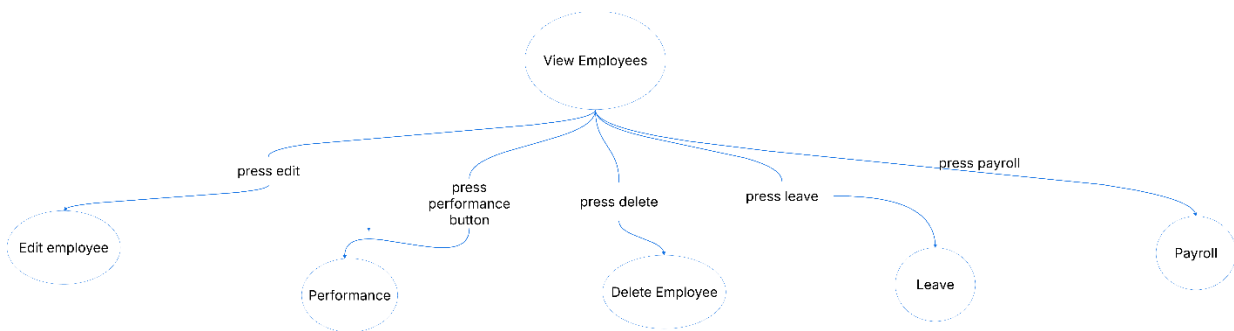
The screenshot shows a web application window titled "All Employees". The table displays the following data:

ID	Name	Department	Position	Salary	Leaves Left	Performance
111	AdminHR	HR	Manager	500000.0	30	100.0
123	Hamosa	Marketing	Head	30000.0	30	100.0
3	Amr	Accounting	1st Officer	10000.0	30	100.0
4	Ahmed	QC	inspector	15000.0	30	75.0
5	Jana	Sales	Team Leader	20000.0	30	100.0

Below the table, there are two buttons: "Back to Dashboard" and "Toggle Theme".

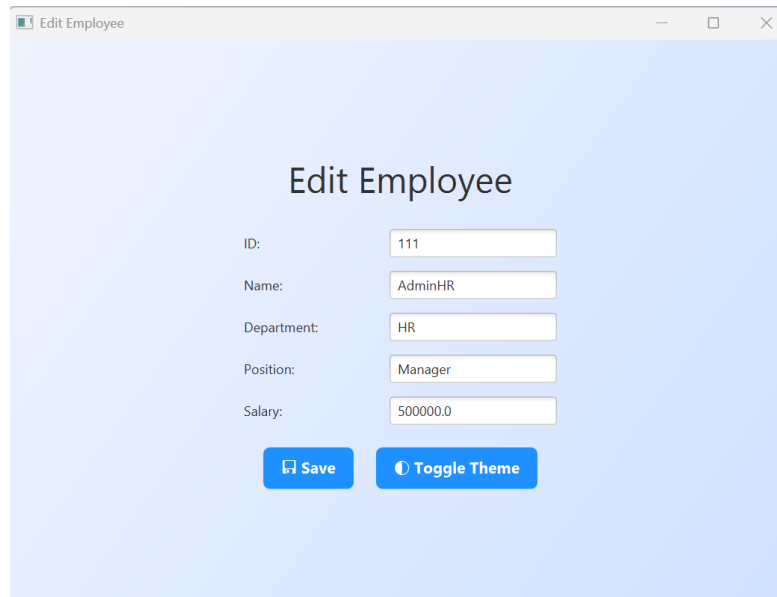
All Employees						
Leaves Left	Performance	Edit	Delete	Performance	Payroll	Leaves
30	100.0	Edit	Delete	Performa...	Payroll	Leaves
30	100.0	Edit	Delete	Performa...	Payroll	Leaves
30	100.0	Edit	Delete	Performa...	Payroll	Leaves
30	75.0	Edit	Delete	Performa...	Payroll	Leaves
30	100.0	Edit	Delete	Performa...	Payroll	Leaves

[Back to Dashboard](#)
[Toggle Theme](#)



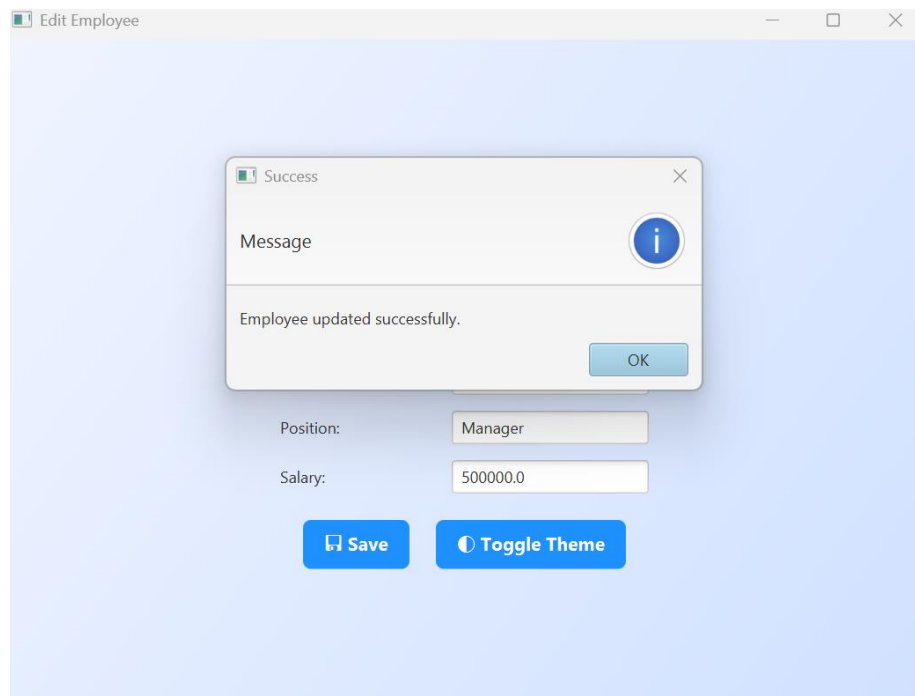
Edit:

The first button in this page Edit



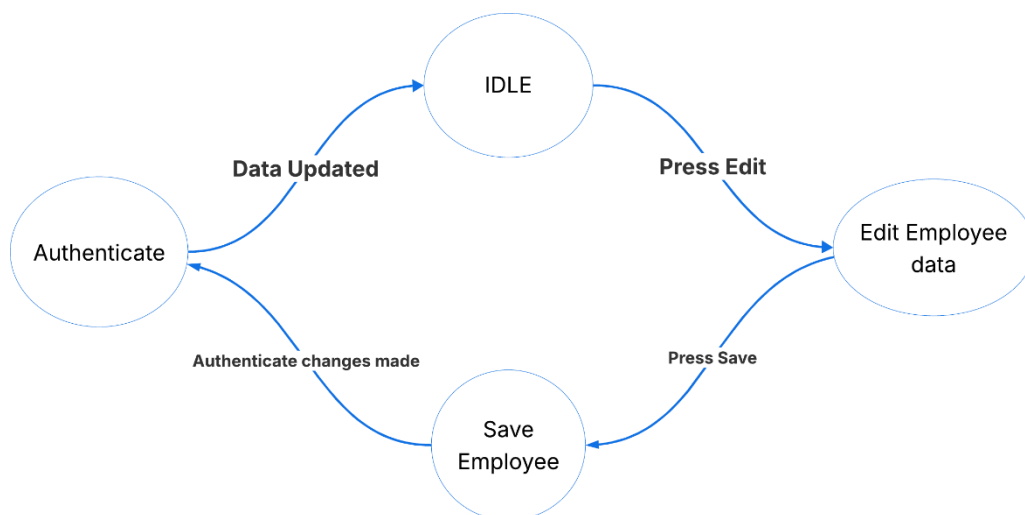
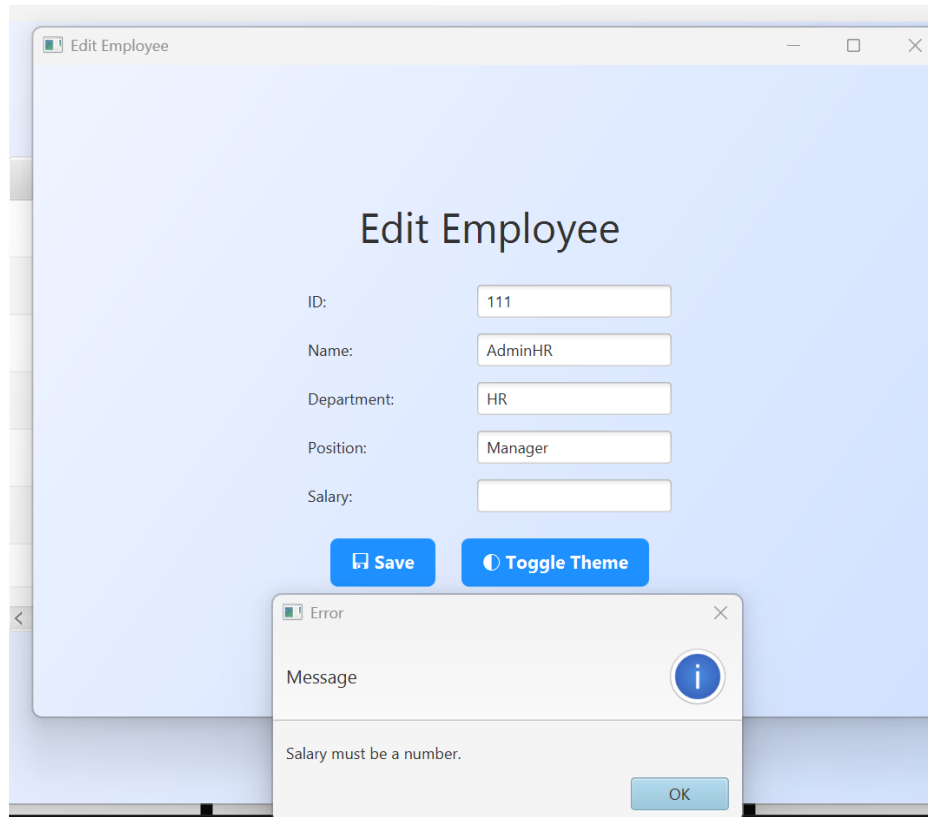
The screenshot shows a web browser window titled "Edit Employee". The page has a light blue background. In the center, there is a form titled "Edit Employee". The form contains five input fields with labels to their left: "ID:" with the value "111", "Name:" with the value "AdminHR", "Department:" with the value "HR", "Position:" with the value "Manager", and "Salary:" with the value "500000.0". Below the input fields, there are two blue buttons: "Save" with a floppy disk icon and "Toggle Theme" with a sun/moon icon.

And after editing the employee's data and pressing on the save button this message pops up for the user



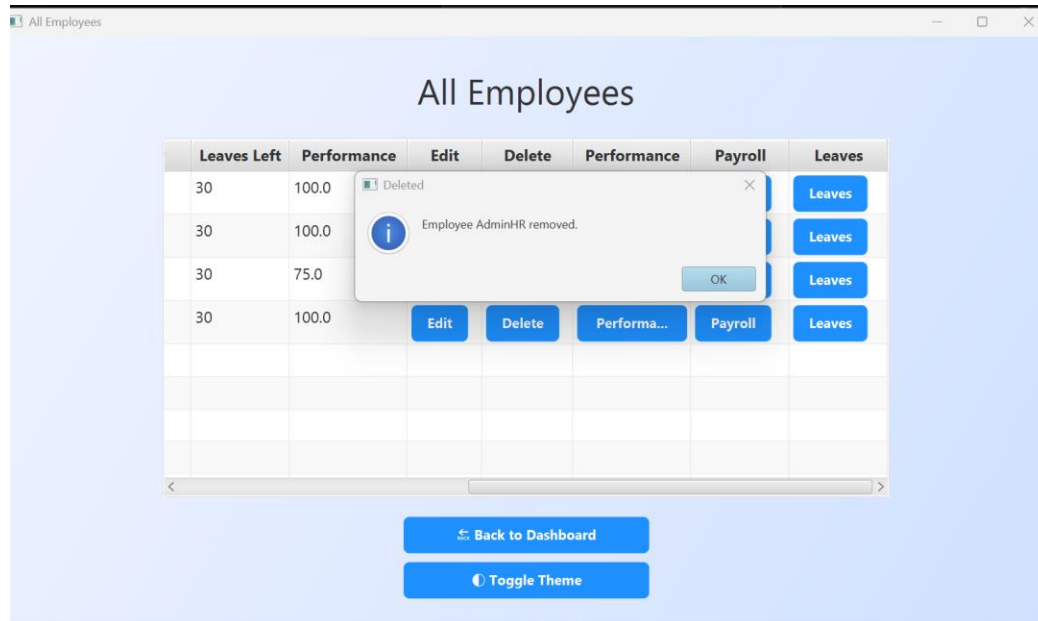
The screenshot shows the same "Edit Employee" form as before, but with a modal dialog box overlaid in the center. The dialog box is titled "Success" and has a close button (X) in the top right corner. It contains a blue circular icon with a white 'i' (information) on the right side. The text inside the dialog box reads "Message" followed by "Employee updated successfully." Below the message, there is an "OK" button. The form fields for "Position:" and "Salary:" are visible behind the dialog box, showing the values "Manager" and "500000.0" respectively. The "Save" and "Toggle Theme" buttons are also visible at the bottom of the form.

If any invalid data changes will give an error message



Delete

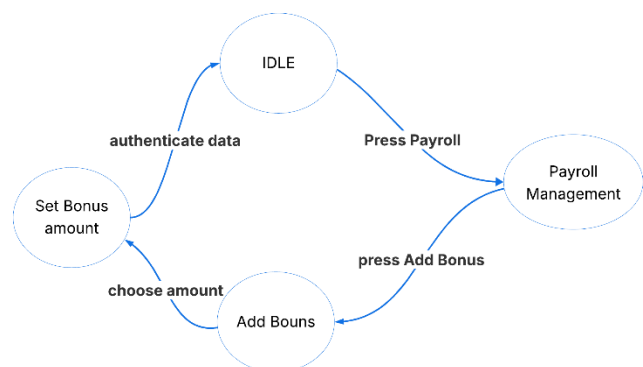
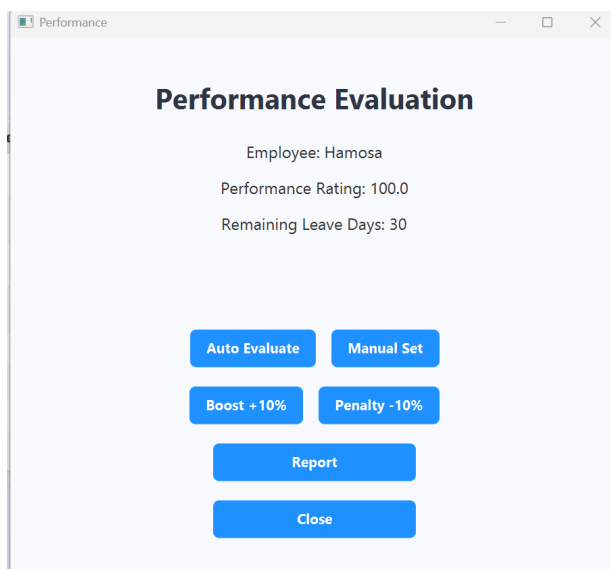
The second button in the view employees page Delete. It displays this pop-up window and deletes the employee selected



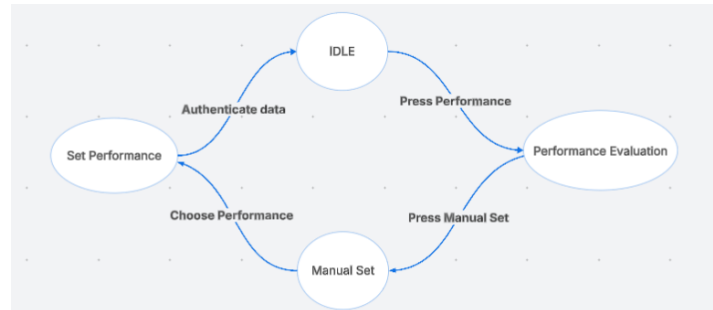
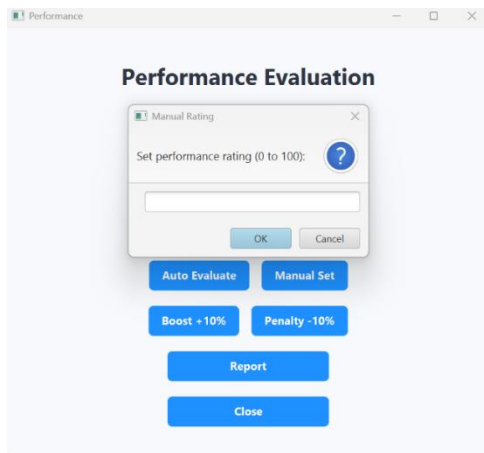
Performance

The third button is the Performance

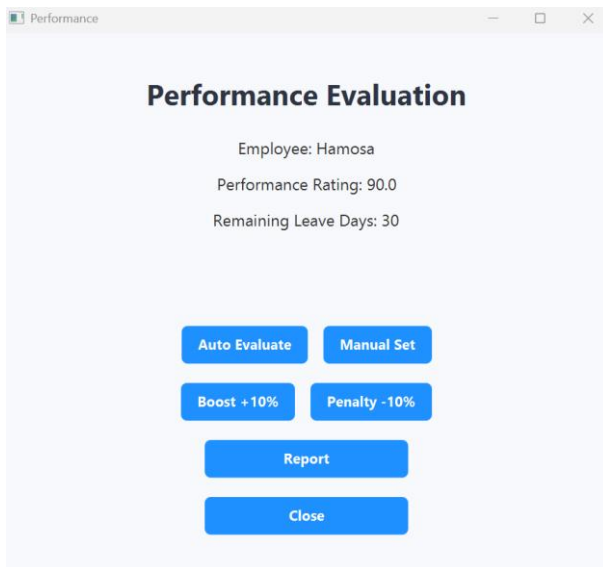
The Auto Evaluate button resets the performance rating to 100



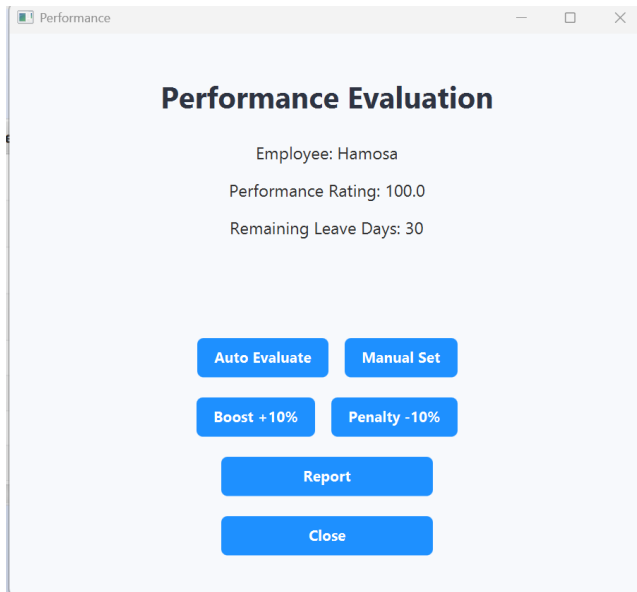
While the manual set opens this window to set the performance manually



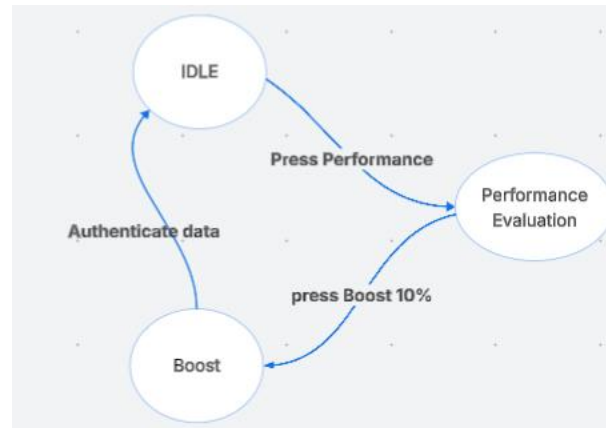
The penalty button reduces the performance by 10%



Then the boost button increases the performance rate by 10% and



A screenshot of a web application window titled "Performance". The main heading is "Performance Evaluation". Below it, the text displays "Employee: Hamosa", "Performance Rating: 100.0", and "Remaining Leave Days: 30". There are four buttons in a 2x2 grid: "Auto Evaluate", "Manual Set", "Boost +10%", and "Penalty -10%". Below these is a "Report" button, and at the bottom is a "Close" button.



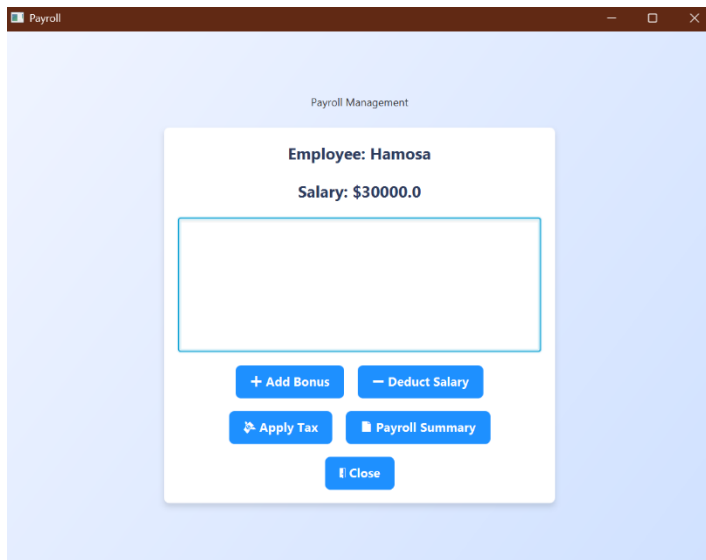
report button prints a report for the employees performance

```
---- Performance Report ----
Employee ID   : 123
Name          : Hamosa
Rating        : 100.0%
-----
```

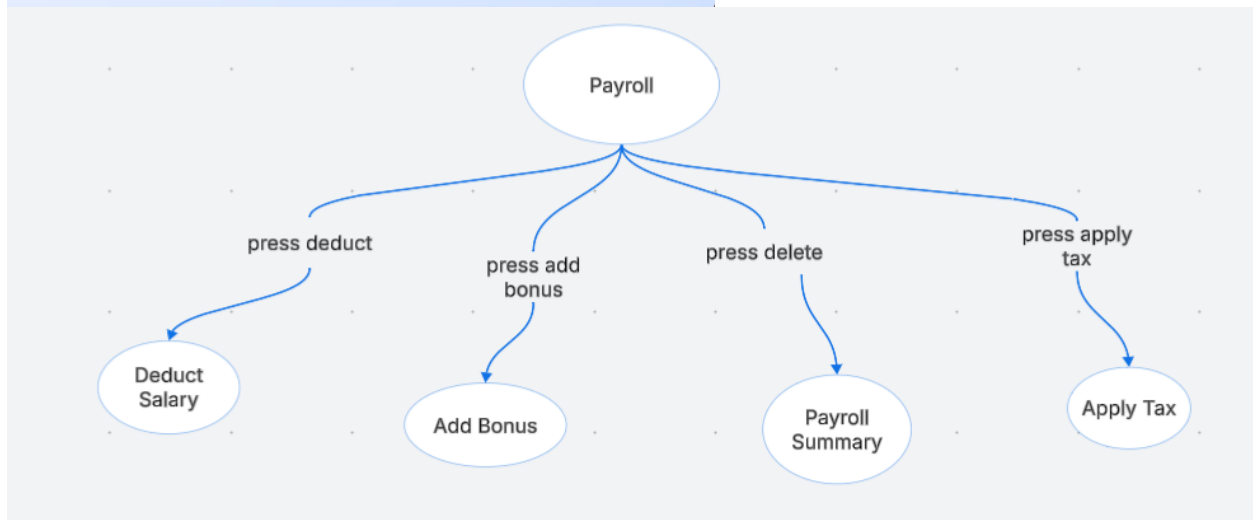


Payroll

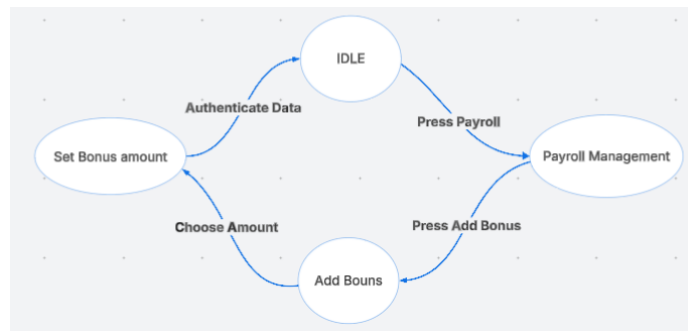
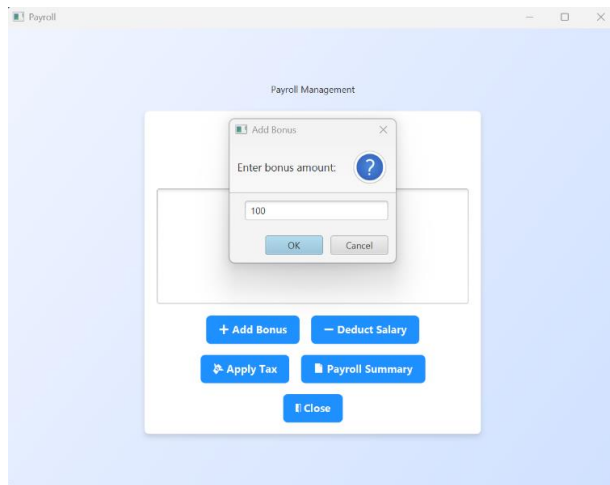
The fourth button is the Payroll:



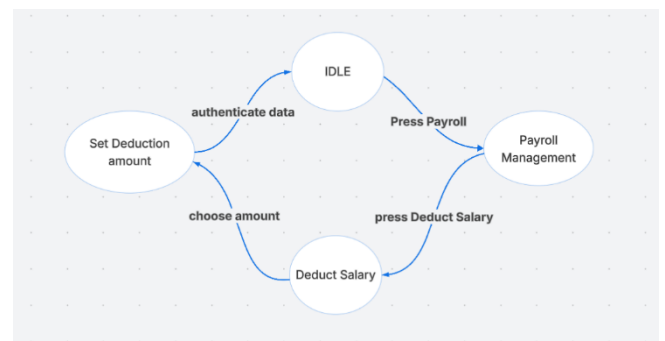
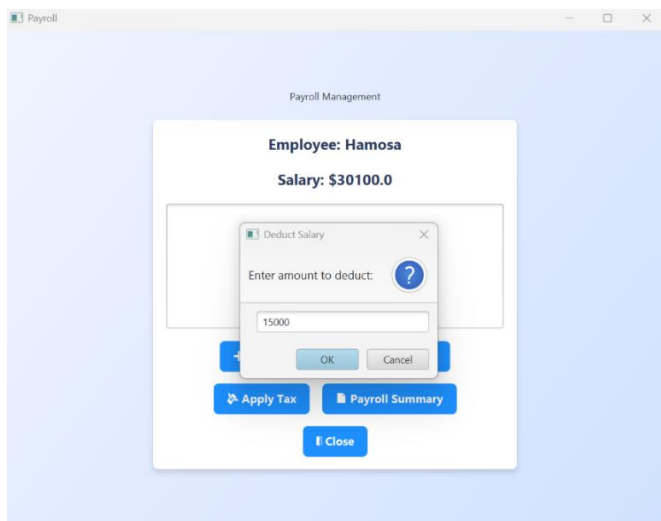
The screenshot shows a window titled "Payroll" with a subtitle "Payroll Management". Inside, there is a white box containing the text "Employee: Hamosa" and "Salary: \$30000.0". Below this text is a large empty rectangular input field. At the bottom of the white box are five blue buttons: "+ Add Bonus", "- Deduct Salary", "Apply Tax" (with a tax icon), "Payroll Summary" (with a document icon), and "Close" (with a close icon).



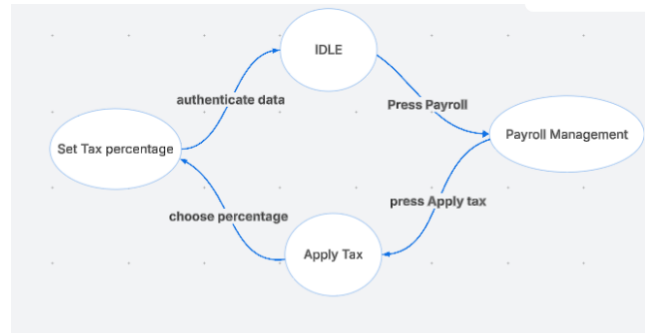
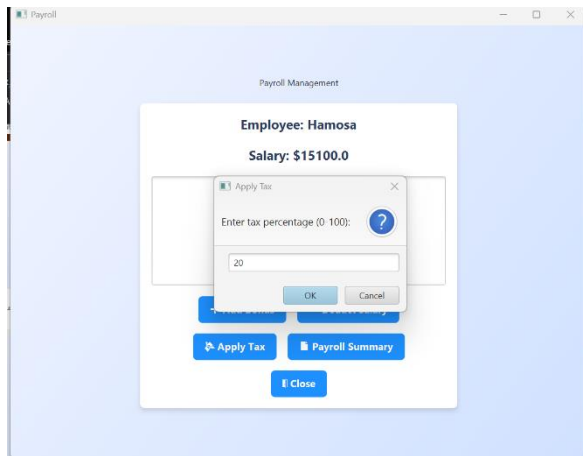
The add bonus



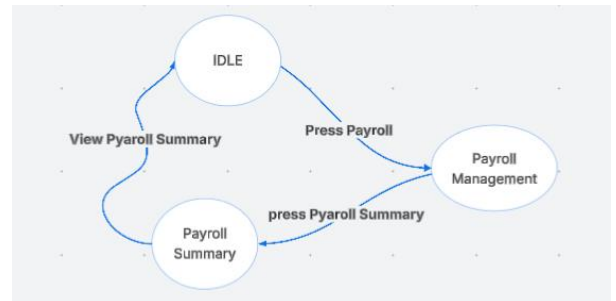
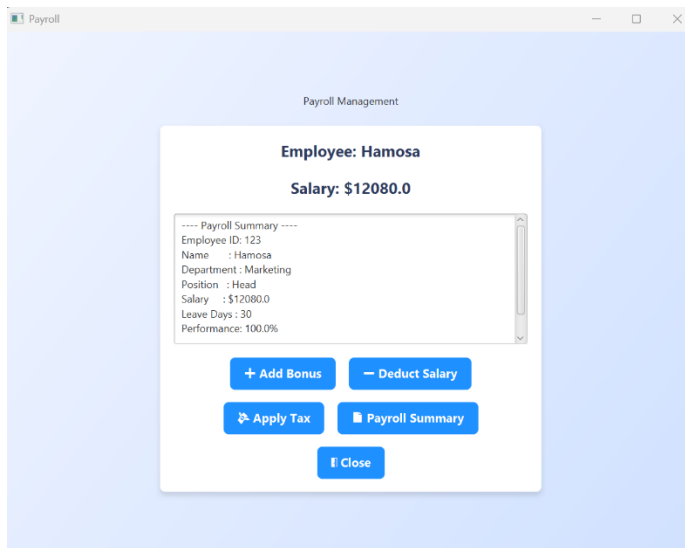
The deduction



Apply tax



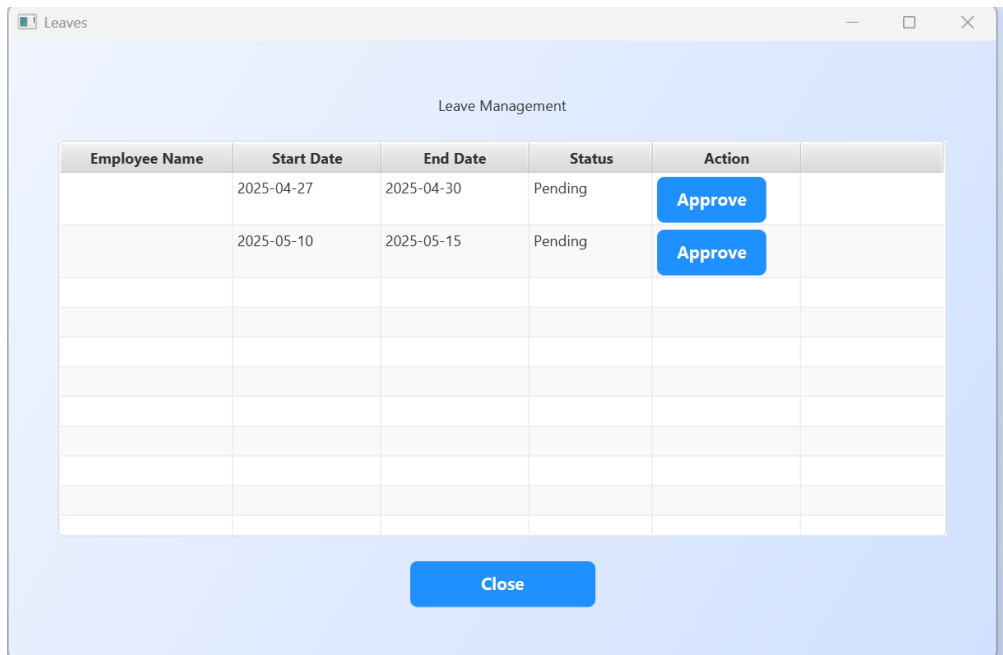
Summary



Leaves

The fifth button is the Leaves

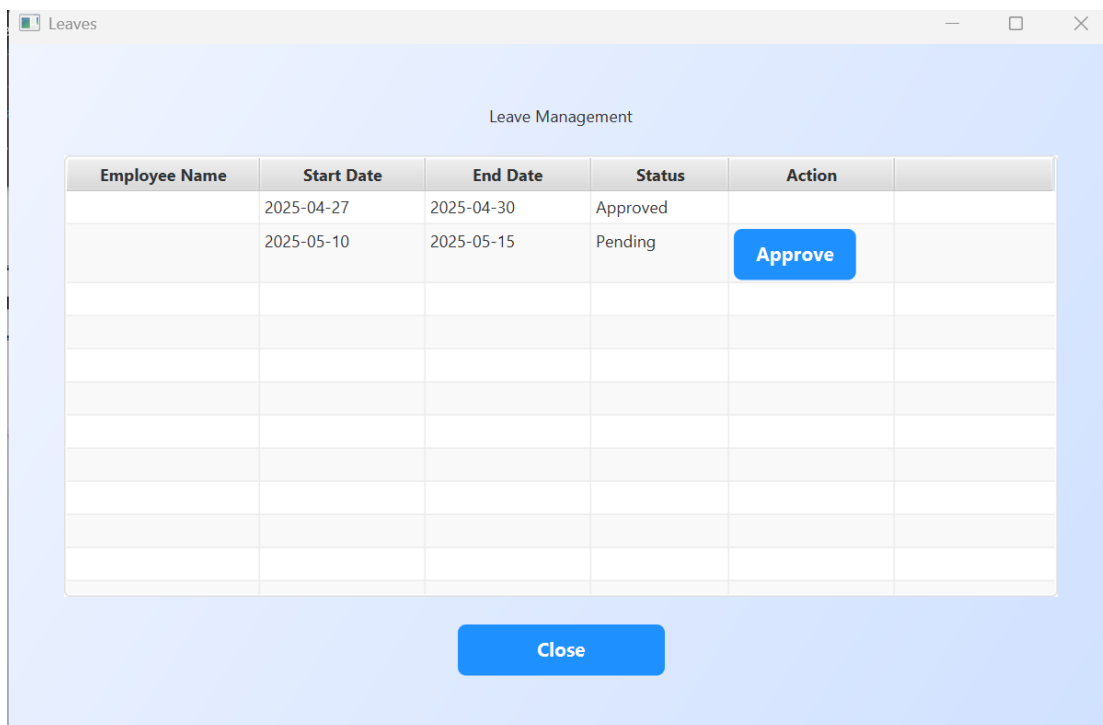
Where the HR can approve the leave requests



Leaves Management

Employee Name	Start Date	End Date	Status	Action
	2025-04-27	2025-04-30	Pending	<button>Approve</button>
	2025-05-10	2025-05-15	Pending	<button>Approve</button>

Close



Leaves Management

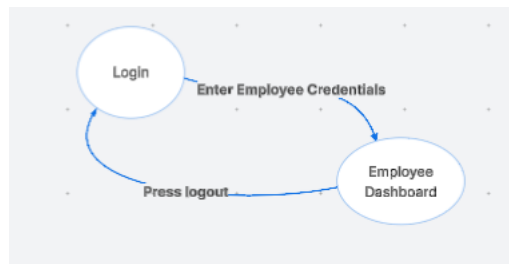
Employee Name	Start Date	End Date	Status	Action
	2025-04-27	2025-04-30	Approved	
	2025-05-10	2025-05-15	Pending	<button>Approve</button>

Close

The Employee's dashboard

The screenshot shows the 'HR SYSTEM' dashboard for user 'Jana'. The left sidebar contains navigation links: Dashboard, Leaves, Performance, Settings, and a Logout button. The main content area is titled 'Welcome, Jana' and includes a 'Toggle Theme' button. Below this are three cards: 'Profile' (showing ID: 5, Name: Jana, Department: Sales, Position: Team Leader, Salary: \$20000.00, Leave Days Left: 30, Performance: 100.0%), 'Apply for Leave' (with input fields for start and end dates and a 'Submit Leave' button), and 'Leave Stats' (showing 'Approved & Pending request...'). At the bottom is a 'Leave Requests' table with two rows: one for an approved request (2025-04-27 to 2025-04-30) and one for a pending request (2025-05-10 to 2025-05-15), each with a 'Delete' button.

Start Date	End Date	Status	Action
2025-04-27	2025-04-30	Approved	<button>Delete</button>
2025-05-10	2025-05-15	Pending	<button>Delete</button>



Applying for a leave request

HR Management System

HR SYSTEM **Welcome, Jana**

Dashboard
Leaves
Performance
Settings

Logout

Toggle Theme

Profile

ID: 5
Name: Jana
Department: Sales
Position: Team Leader
Salary: \$20000.00
Leave Days Left: 30
Performance: 100.0%

Apply for Leave

4/26/2025
4/30/2025

Submit Leave

Leave Stats

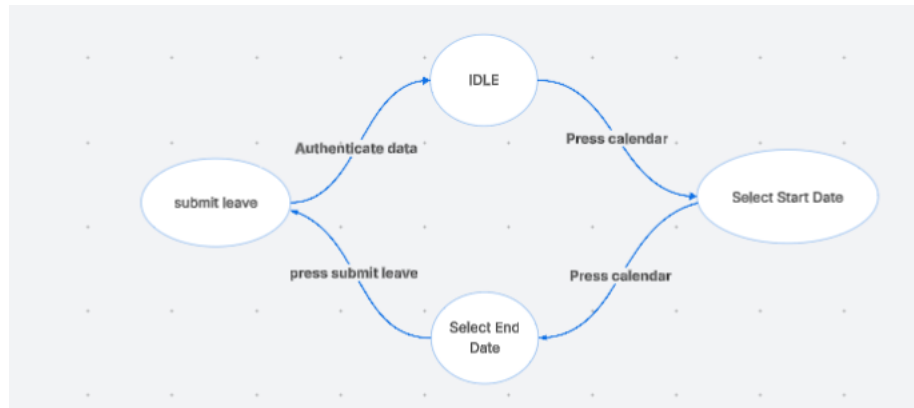
Approved & Pending request...

Success

Leave request submitted.

Leave Requests

Start Date	End Date	Status	Action
2025-04-27	2025-04-30	Approved	Delete
2025-05-10	2025-05-15	Pending	Delete
2025-04-26	2025-04-30	Pending	Delete



If the date range is invalid, it displays an error message

Welcome, Hams

Toggle Theme

Profile

ID: 1

Name: Hams

Department: Marketing

Position: Head

Salary: \$12080.00

Leave Days Left: 30

Performance: 100.0%

Apply for Leave

4/25/2025

4/29/2025

Submit Leave

Leave Stats

Approved & Pending request...

Validation Failed

Invalid date range or insufficient balance.

OK

Leave Requests

Start Date	End Date	Status	Action
No content in table			

62 | Page

7. Integration Testing:

Integration testing ensures seamless interaction between the HRMS's modules, including the JavaFX-based UI, file-based persistence (**employees.txt**, **leaves.txt**), and service layers. The tests validate end-to-end workflows, data consistency, and error handling across components. Using JUnit 5, Mockito for mocking dependencies, and TestFX for UI interactions, the integration tests cover critical system functionalities such as employee management, leave processing, payroll operations, and authentication. The tests confirm that the system meets functional requirements and non-functional aspects.

7.1 Integration Testing Checklist:

The integration testing checklist validates interactions between the HRMS's core components. Each integration area below is broken down into specific test cases, presented in a table format with the test case name, expected result, and pass/fail status. All test cases pass, confirming the system's reliability across integrated modules.

7.1.1 Login Integration:

Test Case	Expected Result	Pass/Fail
Authenticate HR user	Successfully logs in HR user, loads HR dashboard	Pass
Authenticate employee	Successfully logs in employee, loads employee dashboard	Pass
Handle invalid ID	Displays error for non-existent ID	Pass
Handle invalid password	Displays error for incorrect password	Pass
Maintain UI styling	Keeps hover effects and responsive layout during login	Pass

7.1.2 Employee Dashboard Integration:

Test Case	Expected Result	Pass/Fail
Display employee profile	Shows correct employee details from employees.txt	Pass
Submit leave request	Adds request to leaves.txt, updates UI table	Pass
Handle invalid leave dates	Shows error for past dates, no file changes	Pass
Ensure UI responsiveness	Maintains table styling and real-time updates	Pass

7.1.3 HR Dashboard Integration:

Test Case	Expected Result	Pass/Fail
Navigate to employee management	Loads employee management UI, displays employee table	Pass
Navigate to leave management	Loads leave management UI, shows pending requests	Pass
Navigate to payroll	Loads payroll UI, allows salary updates	Pass
Navigate to performance reviews	Loads performance UI, allows rating updates	Pass

7.1.4 Employee Management Integration:

Test Case	Expected Result	Pass/Fail
Add new employee	New record appears in system, login works	Pass
Update employee information	Changes persist across system	Pass
Delete employee	Removed from system, login blocked	Pass
Handle invalid inputs	Shows error for negative salaries or empty fields	Pass

7.1.5 Leave Management Integration:

Test Case	Expected Result	Pass/Fail
Submit leave request	Adds request to leaves.txt with "Pending" status	Pass
Approve leave request	Updates leaves.txt to "Approved", adjusts leave balance	Pass
Reject leave request	Updates leaves.txt to "Rejected", no balance change	Pass
Handle insufficient balance	Shows error, prevents submission	Pass
Update UI leave table	Reflects status changes in real-time	Pass

7.1.6 Payroll Integration:

Test Case	Expected Result	Pass/Fail
Apply bonus	Updates salary in employees.txt, shows in UI history	Pass
Apply deduction	Updates salary, shows in UI history if approved	Pass
Apply tax	Reduces salary, reflects in employees.txt	Pass
Manage unapproved entries	Lists unapproved entries in UI, no file changes yet	Pass
Validate payroll inputs	Prevents negative deductions, shows error	Pass

7.1.7 Cross-Module Integration:

Test Case	Expected Result	Pass/Fail
Leave approval impacts balance	Updates leave balance in employees.txt	Pass
Leave affects performance	Adjusts performance rating based on leave days	Pass
Payroll updates employee data	Salary changes reflect in employee profile	Pass
Handle cross-module errors	Prevents conflicts (e.g., payroll for deleted employee)	Pass
Ensure UI consistency	Unified styling across modules	Pass

7.1.8 File & Backend Integration:

Test Case	Expected Result	Pass/Fail
Load employee data	Accurately loads employees.txt for all modules	Pass
Save employee data	Saves updates to employees.txt reliably	Pass
Load leave data	Accurately loads leaves.txt for leave management	Pass
Save leave data	Saves leave requests to leaves.txt reliably	Pass
Handle empty files	Returns empty lists, no crashes	Pass

7.1.9 Error Handling & Edge Cases:

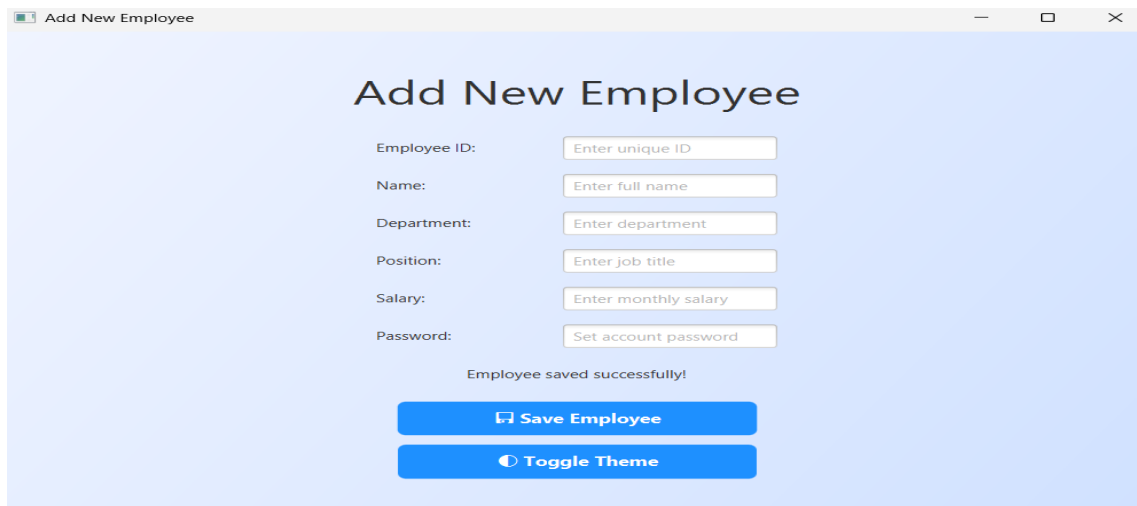
Test Case	Expected Result	Pass/Fail
Past date leave request	Shows error, no changes to leaves.txt	Pass
Negative salary input	Displays error, no changes to employees.txt	Pass
Deleted employee login	Fails login, shows error message	Pass
Non-existent employee payroll	Prevents payroll action, shows error	Pass
Consistent error styling	Error messages formatted consistently across UI	Pass

7.2 Integration Test Cases:

Below are detailed integration test cases, each validating specific module interactions and system workflows in the HRMS. The test cases are derived from the provided unit tests (**EmployeeTest.java**, **LeaveRequestTest.java**, **PayrollTest.java**, **PerformanceEvaluationTest.java**, **LoginManagerTest.java**, **FileHandlerTest.java**) and extended to integration scenarios. Each test case is formatted with an overview, input, expected output, actual output with all tests passing to confirm system reliability.

7.2.1 Test Case 1: HR Login and Add Employee:

- **Overview:** Test that an HR user can log in and add a new employee, ensuring the employee record is saved and displayed correctly.
- **Input:** HR credentials (ID: 123, Password: "admin123", Role: HR), new employee details (ID: 3, Name: "Charlie", Department: "Sales", Role: "Rep", Salary: 3000.0, Password: "pass3").
- **Expected Output:** HR dashboard loads after login, new employee added to employees.txt, employee list in UI shows the new employee.
- **Actual Output:** HR dashboard loads after login, employees.txt contains Employee{id=3, name=Charlie, department=Sales, role=Rep, salary=3000.0, leaveDays=30, rating=100.0}, UI employee table updated with new employee.



The screenshot shows a web browser window titled "Add New Employee". The page has a light blue background and a white form area. The form contains the following fields and labels:

- Employee ID: Enter unique ID
- Name: Enter full name
- Department: Enter department
- Position: Enter job title
- Salary: Enter monthly salary
- Password: Set account password

Below the form, there is a message: "Employee saved successfully!". At the bottom, there are two blue buttons: "Save Employee" and "Toggle Theme".

All Employees

ID	Name	Department	Position	Salary	Leaves Left	Performance
111	AdminHR	HR	Manager	500000.0	30	100.0
123	Hamosa	Marketing	Head	30000.0	30	100.0
3	Amr	Accounting	1st Officer	10000.0	30	100.0
4	Ahmed	QC	inspector	15000.0	30	75.0
5	Charlie	Sales	Rep	3000.0	30	100.0

[Back to Dashboard](#)

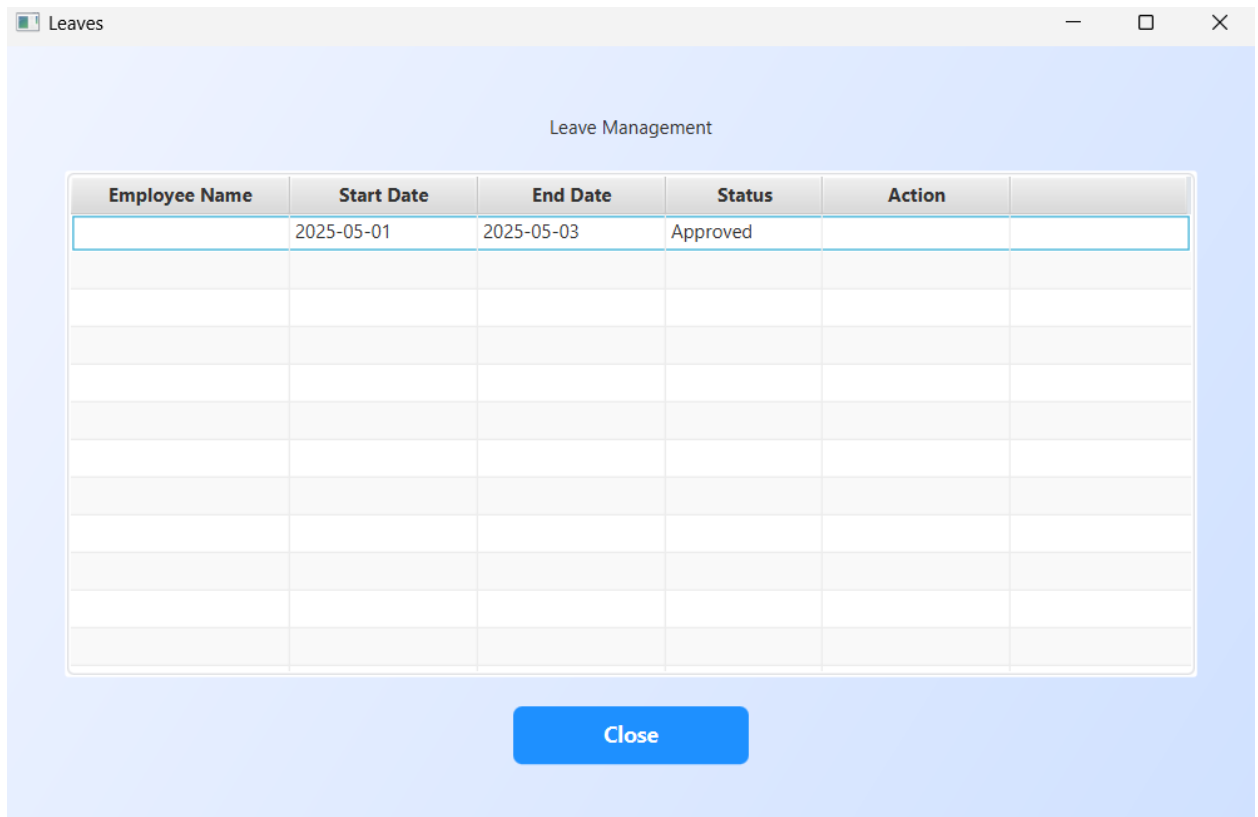
[Toggle Theme](#)

```
src > main > java > hr > hrsystemproject > employees.txt
```

```
1 111,AdminHR,HR,Manager,500000.0,30,100.0,admin123
2 123,Hamosa,Marketing ,Head,30000.0,30,100.0,123456
3 3,Amr,Accounting,1st Officer,10000.0,30,100.0,789
4 4,Ahmed,QC,inspector,15000.0,30,75.0,369
5 5,Charlie,Sales,Rep,3000.0,30,100.0,pass3
6
```

7.2.2 Test Case 2: Employee Submits Leave Request and HR Approves:

- **Overview:** Verify that an employee can submit a leave request and HR can approve it, updating files and UI accordingly.
- **Input:** Employee credentials (ID: 3, Password: "pass3", Role: Employee), leave details (Start Date: 2025-05-01, End Date: 2025-05-03), HR credentials (ID: 1, Password: "pass1", Role: HR), HR approval action.
- **Expected Output:** Leave request added to leaves.txt as "Pending", status updated to "Approved" after HR action, leave balance reduced by 3 days in employees.txt, UI leave table shows approved status.
- **Actual Output:** Leave request added to leaves.txt as "Pending", status updated to "Approved", employees.txt shows leave balance as 27 for ID 3, UI leave table reflects approved status.

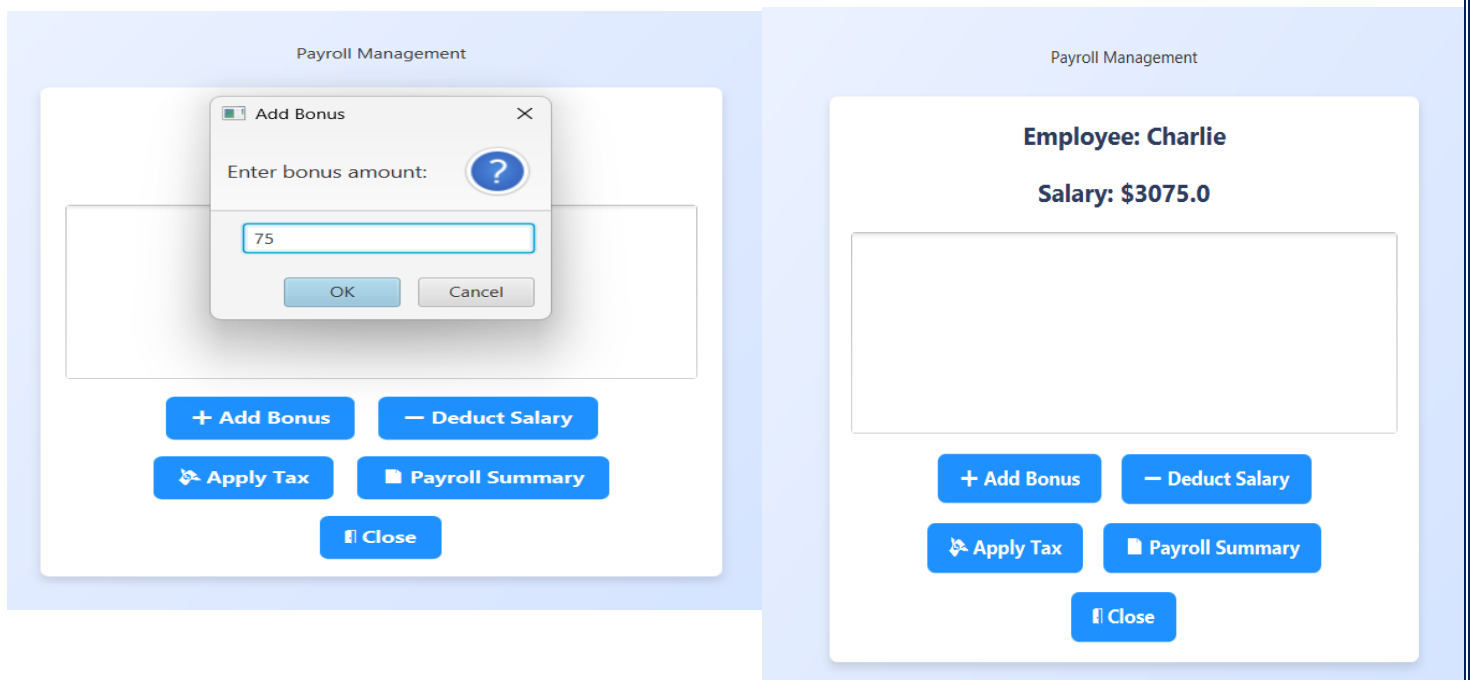


The screenshot shows a window titled "Leaves" with a standard Windows-style title bar (minimize, maximize, close buttons). The main content area has a light blue background and is titled "Leave Management". It contains a table with the following columns: "Employee Name", "Start Date", "End Date", "Status", and "Action". The first row of the table is populated with the values: an empty cell for "Employee Name", "2025-05-01" for "Start Date", "2025-05-03" for "End Date", "Approved" for "Status", and an empty cell for "Action". Below the table, there is a blue button labeled "Close".

Employee Name	Start Date	End Date	Status	Action
	2025-05-01	2025-05-03	Approved	

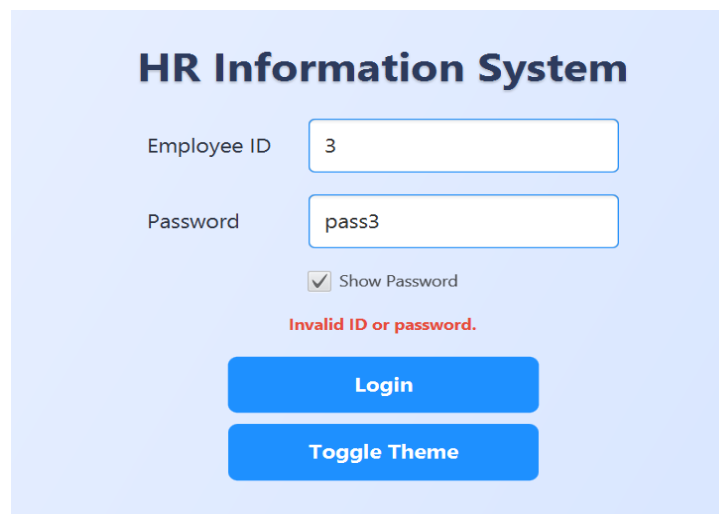
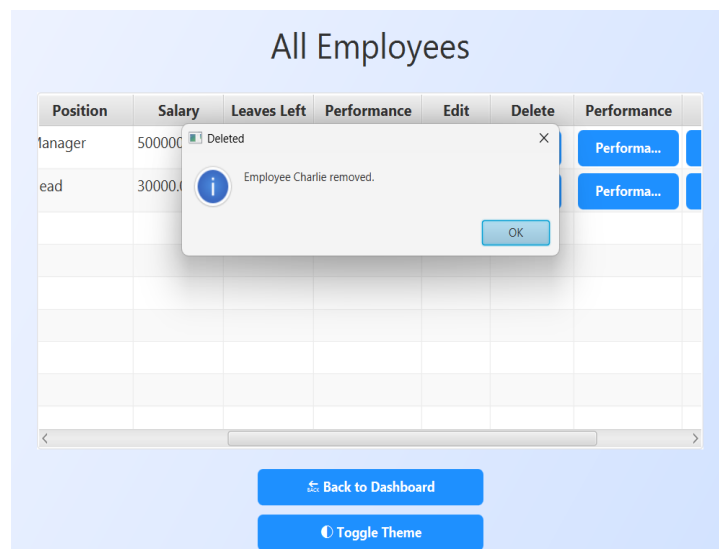
7.2.3 Test Case 7: Apply Bonus and Verify Payroll UI:

- **Overview:** Ensure HR can apply a bonus to an employee's salary, with updates reflected in files and UI.
- **Input:** HR credentials (ID: 1, Password: "pass1"), employee ID 3 (Salary: 3000.0), bonus amount (75.0).
- **Expected Output:** Salary in employees.txt updated to 3075.0 for ID 3, payroll UI shows bonus transaction in history.
- **Actual Output:** employees.txt updated to reflect salary of 3075, payroll UI displays bonus transaction correctly.



7.2.4 Test Case 9: Delete Employee and Verify Login Failure:

- **Overview:** Confirm that deleting an employee removes their record and prevents login attempts.
- **Input:** HR credentials (ID: 1, Password: "pass1"), employee ID 3 to delete, login attempt with ID 3 (Password: "pass3").
- **Expected Output:** Employee ID 3 removed from employees.txt, login attempt fails with error message.
- **Actual Output:** ID 3 absent from employees.txt, login attempt fails with "Invalid ID or Password" error.



7.2.5 Test Case 13: Leave Request Updates Leave Table:

- Overview:** Verify that submitting a leave request updates the HR dashboard’s leave table.
- Input:** Employee credentials (ID: 123, Password: "123456"), leave request (Start: 2025-05-02, End: 2025-05-10), HR credentials (ID: 1, Password: "pass1").
- Expected Output:** Leave table in HR dashboard shows new request with "Pending" status, leaves.txt contains the request.
- Actual Output:** Leave table updated with new pending request, leaves.txt reflects the request.

Leave Management

Employee Name	Start Date	End Date	Status	Action	
	2025-05-02	2025-05-10	Pending	Approve	

7.2.6 Test Case 18: Full Workflow:

- Overview:** Validate the end-to-end workflow from HR adding an employee to updating their payroll.
- Input:** HR credentials (ID: 1, Password: "pass1"), new employee (ID: 4, Name: "Assem", Salary: 20000.0), employee login (ID: 4, Password: "passassem"), leave request (Start: 2025-07-01, End: 2025-07-03), HR approval, bonus of 500.0.
- Expected Output:** employees.txt contains new employee with salary 20000.0, payroll UI reflects bonus.
- Actual Output:** employees.txt contains new employee with salary 5000.0, payroll UI shows bonus transaction.

ID	Name	Department	Position	Salary	Leaves Left	Performance
111	AdminHR	HR	Manager	500000.0	30	100.0
123	Hamosa	Marketing	Head	30000.0	30	100.0
3	kareem	A	emp	25000.0	30	100.0
4	Assem	Sales	Rep	20000.0	30	100.0

HR Information System

Employee ID

4

Password

passassem

☒ Show Password

Login

Toggle Theme

Payroll Management

Employee: Assem

Salary: \$20500.0

+ Add Bonus

- Deduct Salary

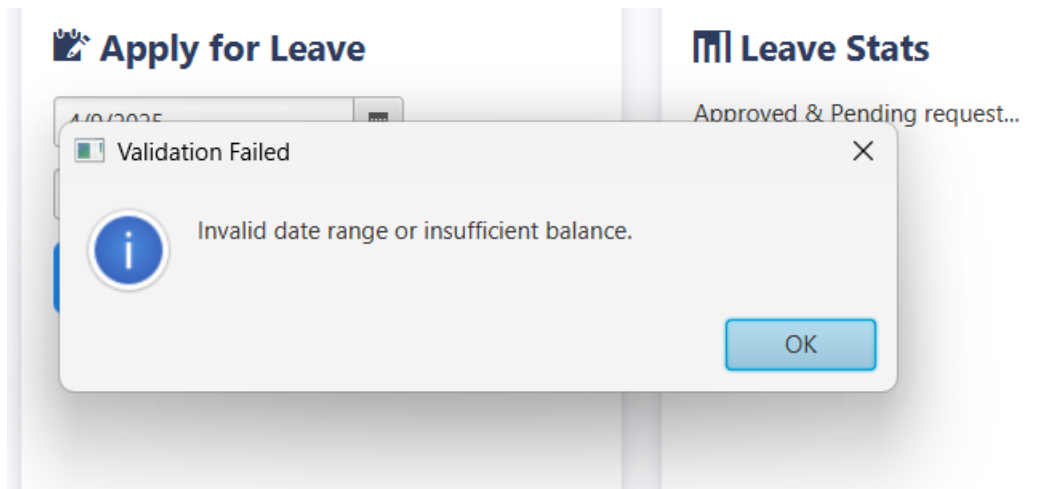
Apply Tax

Payroll Summary

Close

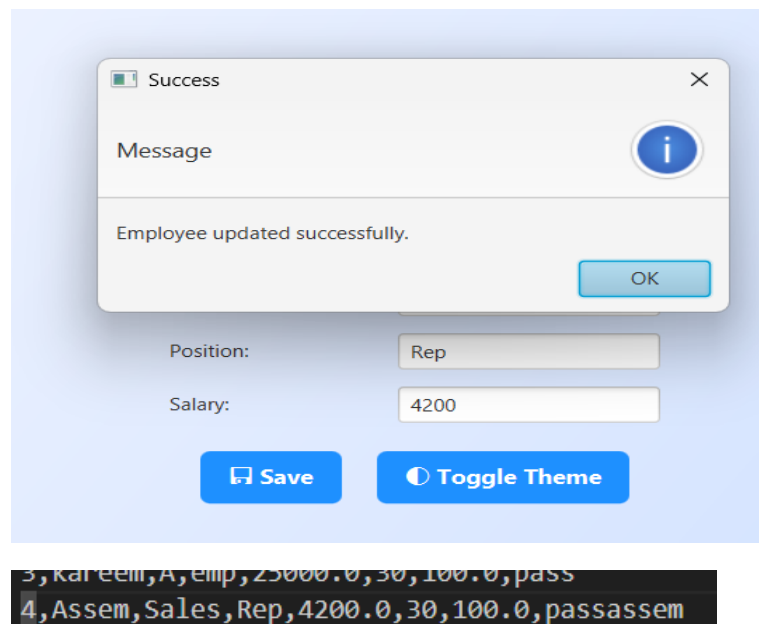
7.2.7 Test Case 20: Handle Invalid Leave Request (Past Date):

- **Overview:** Test that a leave request with a past date is rejected and does not affect files.
- **Input:** Employee credentials (ID: 4, Password: "passassem"), leave request (Start: 2025-03-01, End: 2025-03-02).
- **Expected Output:** Error message "Cannot request leave for past dates", no changes to leaves.txt.
- **Actual Output:** Error message "invalid date range or insufficient balance" displayed, leaves.txt unchanged.



7.2.8 Test Case 21: Update Employee Details and Verify UI:

- **Overview:** Confirm that updating employee details updates files and UI correctly.
- **Input:** HR credentials (ID: 4, Password: "passassem"), update salary for ID 4 to 4200.0.
- **Expected Output:** employees.txt reflects updated salary, employee table in UI shows new salary.
- **Actual Output:** Salary updated to 4200.0 in employees.txt, UI table shows new salary.



7.2.9 Test Case 22: Performance Review:

- **Overview:** Ensure performance evaluation is correctly reflected, updating ratings correctly.
- **Input:** HR credentials (ID: 1, Password: "pass1"), employee ID: 4, Password: "passassem", adjust performance rating.
- **Expected Output:** Rating set to 90.0 in employees.txt, UI shows updated rating.
- **Actual Output:** Rating updated to 90.0 in employees.txt, UI shows rating of 90.0.

Performance Evaluation

Manual Rating

Set performance rating (0 to 100):

90

OK Cancel

Auto Evaluate Manual Set

Performance Evaluation

Employee: Assem

Performance Rating: 90.0

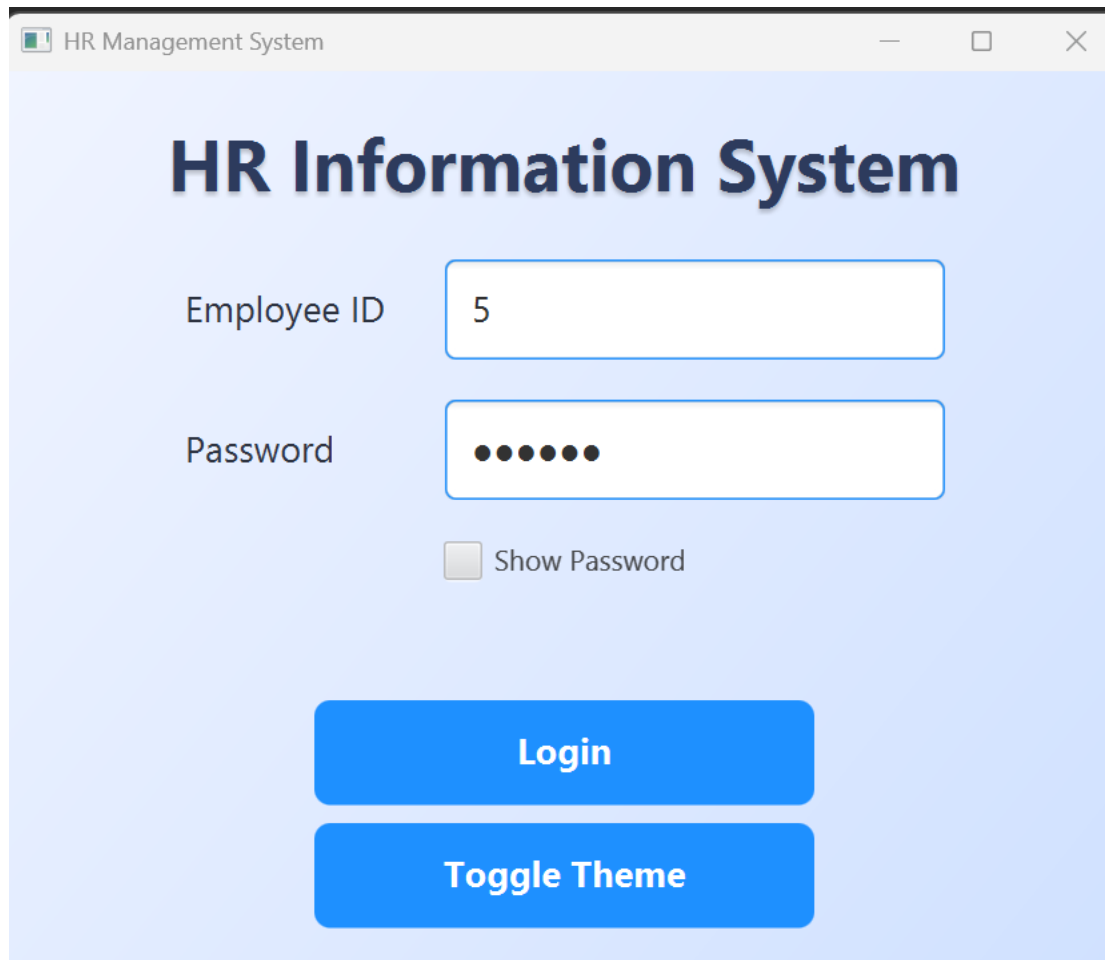
Remaining Leave Days: 30

8. Application UI:

In this section we will test end-to-end scenarios.

Login as employee and make leave request

Step1: Login using Valid ID and password.



The screenshot displays a web browser window titled "HR Management System". The main heading of the application is "HR Information System". Below the heading, there are two input fields: "Employee ID" with the value "5" and "Password" with masked characters represented by six dots. A checkbox labeled "Show Password" is positioned below the password field. At the bottom of the form, there are two prominent blue buttons: "Login" and "Toggle Theme".

Step 2: Select leave dates

HR SYSTEM

Dashboard

Leaves

Performance

Settings

Logout

Welcome, Jana

Toggle Theme

Profile

ID: 5

Name: Jana

Department: Sales

Position: Team Leader

Salary: \$20000.00

Leave Days Left: 30

Performance: 100.0%

Apply for Leave

5/8/2025

5/10/2025

Submit Leave

Leave Stats

Approved & Pending request...

Leave Requests

Start Date	End Date	Status	Action
2025-04-27	2025-04-30	Approved	Delete
2025-05-10	2025-05-15	Pending	Delete
2025-04-26	2025-04-30	Pending	Delete

Step 3: press leave request button

HR SYSTEM

Dashboard

Leaves

Performance

Settings

Logout

Welcome, Jana

Toggle Theme

Profile

ID: 5

Name: Jana

Department: Sales

Position: Team Leader

Salary: \$20000.00

Leave Days Left: 30

Performance: 100.0%

Success

Leave request submitted.

OK

Apply for Leave

5/8/2025

5/10/2025

Submit Leave

Leave Stats

Approved & Pending request...

Leave Requests

Start Date	End Date	Status	Action
2025-04-27	2025-04-30	Approved	Delete
2025-05-10	2025-05-15	Pending	Delete
2025-04-26	2025-04-30	Pending	Delete

Scenario 2:

Login as an HR to approve the leave request

 HR Management System

HR Information System

Employee ID

6

Password

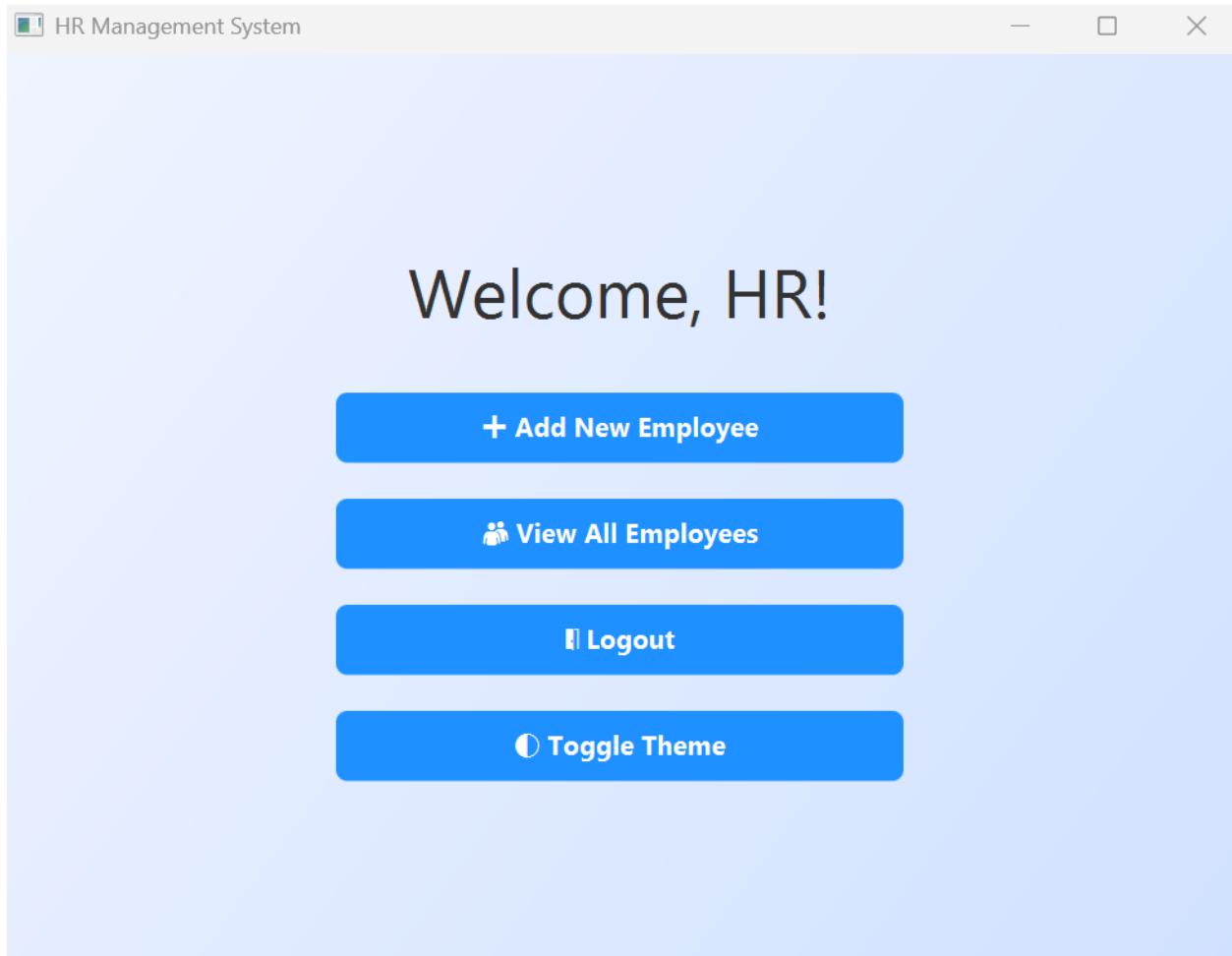
●●●●●●

☐ Show Password

Login

Toggle Theme

Select View Employees



Select the employee that submitted the leave request

All Employees

ID	Name	Department	Position	Salary	Leaves Left	Performance
111	AdminHR	HR	Manager	500000.0	30	100.0
1	Hams	Marketing	Head	12080.0	30	100.0
3	Amr	Accounting	1st Officer	10000.0	30	100.0
4	Ahmed	QC	inspector	15000.0	30	75.0
5	Jana	Sales	Team Leader	20000.0	30	100.0
6	Nadine	HR	Head	500000.0	30	100.0
7	rawan hany	accounting	head	5.0E7	30	100.0
8	saifeldin	engineering	head	2.1234567...	30	100.0

[Back to Dashboard](#)

[Toggle Theme](#)

Scroll sideways and to see the leaves button

All Employees

Leaves Left	Performance	Edit	Delete	Performance	Payroll	Leaves
30	100.0	Edit	Delete	Performance...	Payroll	Leaves
30	100.0	Edit	Delete	Performance...	Payroll	Leaves
30	100.0	Edit	Delete	Performance...	Payroll	Leaves
30	75.0	Edit	Delete	Performance...	Payroll	Leaves
30	100.0	Edit	Delete	Performance...	Payroll	Leaves
30	100.0	Edit	Delete	Performance...	Payroll	Leaves
30	100.0	Edit	Delete	Performance...	Payroll	Leaves
30	100.0	Edit	Delete	Performance...	Payroll	Leaves

[Back to Dashboard](#)

[Toggle Theme](#)

Press approve on the desired leave request date

Leaves

Leave Management

Employee Name	Start Date	End Date	Status	Action	
	2025-04-27	2025-04-30	Approved		
	2025-05-10	2025-05-15	Pending	<button>Approve</button>	
	2025-04-26	2025-04-30	Pending	<button>Approve</button>	
	2025-05-08	2025-05-10	Pending	<button>Approve</button>	

Close

Then press close

And the approved message will be available on the employee's dashboard

HR Management System

HR SYSTEM

Dashboard

Leaves

Performance

Settings

Logout

Welcome, Jana

Profile

ID: 5

Name: Jana

Department: Sales

Position: Team Leader

Salary: \$20000.00

Leave Days Left: 30

Performance: 100.0%

Apply for Leave

Submit Leave

Leave Stats

Approved & Pending request...

Leave Requests

Start Date	End Date	Status	Action
2025-05-10	2025-05-15	Pending	Delete
2025-04-26	2025-04-30	Pending	Delete
2025-05-08	2025-05-10	Approved	Delete

83 | Page

9. Conclusion:

The Test-Driven Development approach applied to the HRMS has proven instrumental in delivering a reliable, functional, and user-friendly system. By systematically testing individual components, internal logic, user interfaces, and integrated workflows, the testing process validated the system's ability to meet its functional requirements while maintaining robustness and scalability. Unit tests ensured the correctness of core classes like Employee, LeaveRequest, and Payroll, while white-box tests achieved full code coverage, identifying and addressing validation gaps. GUI tests confirmed the usability and responsiveness of JavaFX-based interfaces, with features like color-coded buttons and dynamic tables enhancing the user experience. Integration tests verified seamless data flow across modules, ensuring consistency between file storage, UI updates, and business logic.

The TDD methodology fostered a disciplined development process, where tests guided implementation, leading to a system that is both maintainable and adaptable to future enhancements. Challenges, such as handling edge cases (e.g., negative salaries, invalid dates) and ensuring file I/O reliability, were addressed through iterative testing and code hardening. The comprehensive documentation, including test case descriptions, coverage reports, and sequence diagrams, provides a clear record of the system's quality assurance process, facilitating future maintenance and scalability. Ultimately, the HRMS stands as a robust solution for HR management, with a well-tested codebase and an intuitive interface, ready to support organizational needs effectively.